INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS INTEGRADO AO ENSINO MÉDIO
TURMA 213

RAFAEL GOMES EMILIANO
KAIQUE AVANTE MOREIRA
PYETRO NUNES MARTINS
SAVIO PEREIRA SOARES

SABOR E CLIC: DOCUMENTAÇÃO TÉCNICA DO SISTEMA

São Paulo
2026

RAFAEL GOMES EMILIANO
KAIQUE AVANTE MOREIRA
PYETRO NUNES MARTINS
SAVIO PEREIRA SOARES

SABOR E CLIC: DOCUMENTAÇÃO TÉCNICA DO SISTEMA

Relatório apresentado como atividade do Projeto Integrador da turma 213 do Curso Técnico em Desenvolvimento de Sistemas Integrado ao Ensino Médio do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo.

São Paulo
2026

RESUMO

Este relatório descreve a arquitetura, a implementação e a verificação do Sabor e Clic, sistema de gestão de bancadas e pedidos para espaços gastronômicos. A aplicação usa HTML, CSS e JavaScript na interface, Flask na API e MySQL na persistência. O documento acompanha a operação desde a reserva até a entrega e o encerramento da sessão. Explica o bloqueio de reservas sobrepostas, o cálculo do preço no servidor, a prevenção de pedidos duplicados e o registro transacional do consumo e do estorno de ingredientes. Também descreve autenticação, autorização, relatórios, implantação local e recuperação de dados. Um caso completo pela API, trechos de código e capturas da interface relacionam as regras de negócio às evidências registradas em ambiente de demonstração.

Palavras-chave: reserva de bancadas; gestão de pedidos; controle de estoque; API.

SUMÁRIO

1 INTRODUÇÃO **4**

2 ARQUITETURA E MODELO DO SISTEMA **5**

2.1 Arquitetura e percurso de uma requisição 5

2.2 Perfis, telas e responsabilidades 6

2.3 Modelo de dados e ciclo do atendimento 7

2.4 Bancadas, cardápio e cadastros 8

3 RESERVAS E ATENDIMENTO **9**

3.1 Reserva, conflito de horário e tarifa 9

3.2 Sessão de atendimento e encerramento 10

3.3 Datas, dinheiro e validação de entradas 11

3.4 Carrinho, estado local e envio 12

4 PEDIDOS, ESTOQUE E COZINHA **13**

4.1 Checkout e gravação do pedido 13

4.2 Reenvio, chave de idempotência e concorrência 14

4.3 Ficha técnica e agregação por ingrediente 15

4.4 Substituição, fator e arredondamento 16

4.5 Movimentações, saldo e estorno 17

4.6 KDS, estimativa e estados do preparo 18

5 SEGURANÇA E ACESSO À API **19**

5.1 Login, cookie e sessão revogável 19

5.2 CSRF, propriedade e auditoria 20

5.3 Listas, filtros e relatórios 21

5.4 API e diagnóstico de respostas 22

6 INSTALAÇÃO E VALIDAÇÃO **23**

6.1 Instalação, migração e recuperação 23

6.2 Validação e roteiro de demonstração 24

7 CASO DE PONTA A PONTA PELA API **25**

8 CONCLUSÃO **26**

REFERÊNCIAS **27**

APÊNDICE A – PEDIDO NO PERFIL DO CLIENTE **28**

APÊNDICE B – PEDIDO EM PREPARO NO KDS **29**

APÊNDICE C – GUIA COMPLETO DAS TELAS **30**

1 INTRODUÇÃO

O Sabor e Clic foi desenvolvido no contexto do Projeto Integrador do IFSP. A proposta reúne a reserva de espaços gastronômicos e a operação de pedidos, permitindo que cliente, cozinha e administrador acompanhem o atendimento em um sistema compartilhado (Instituto Federal de São Paulo, 2026).

O objetivo deste relatório é explicar como a implementação integra as telas, as regras de negócio e o banco de dados. A análise segue o código-fonte, o esquema SQL e as suítes de testes do projeto (Sabor e Clic, 2026). Os exemplos numéricos e os identificadores usados no texto são ilustrativos. Os apêndices A, B e C mostram capturas do ambiente de demonstração.

A formatação adota como referência o guia de normalização do IFSP Campus São Paulo (Instituto Federal de São Paulo, [s. d.]).

2 ARQUITETURA E MODELO DO SISTEMA

2.1 Arquitetura e percurso de uma requisição

O Flask entrega as páginas do diretório FrontEnd e recebe as chamadas JSON sob /api. A interface multipágina executa no navegador. O código Python concentra as regras de negócio e usa SQLAlchemy para acessar o MySQL. A mesma origem permite que as páginas e a API compartilhem a sessão autenticada.

Entrada HTTP

Antes de chamar a rota, a aplicação atribui um identificador à requisição, verifica a origem quando informada e aplica a proteção de autenticação. O roteador escolhe o handler pelo caminho e pelo método. Um mesmo caminho pode listar no GET e criar no POST.

Serviço de aplicação e persistência

O handler interpreta o JSON e delega operações como reserva e checkout ao controller operacional. Este carrega os registros que participam da decisão, valida vínculos e executa as alterações na mesma sessão do SQLAlchemy. O MySQL preserva chaves, referências e restrições de integridade. Após a gravação, o handler serializa o resultado.

Resposta ao navegador

A API devolve o recurso ou uma mensagem de erro. O navegador atualiza a tela depois da resposta. Quando a lista é paginada, a API anexa metadados da consulta. X-Request-ID liga a resposta ao registro de auditoria quando o evento é auditado.

Numa confirmação de pedido, o clique chega a views.js, passa pelo transporte de api.js, entra em POST /api/pedidos e termina em checkout(). O carrinho é limpo após a resposta de sucesso.

Arquivos consultados: FrontEnd/assets/api.js e views.js · backend/app/__init__.py · backend/app/routes/api_routes.py

2.2 Perfis, telas e responsabilidades

O atendimento passa por pessoas com responsabilidades distintas. Os menus direcionam cada perfil às telas da sua rotina e a API controla o acesso em cada operação.

Cliente

O cliente consulta bancadas, cria e acompanha suas reservas, inicia a sessão, monta o carrinho e envia o pedido. Pode acompanhar o status e consultar o resumo da própria sessão. Consultas a reservas, sessões e pedidos são limitadas aos vínculos do cliente autenticado.

Cozinha

O cozinheiro acompanha a fila no KDS, informa a estimativa, inicia preparo e registra pronto e entregue. A cozinha pode alterar a disponibilidade de pratos. O preço oficial, a ficha técnica e a configuração de substituições pertencem à gestão administrativa.

Administrador

O administrador mantém usuários, bancadas e catálogo. A configuração da ficha técnica define o consumo esperado, e as regras de substituição definem quais alternativas podem ser usadas. A área administrativa também consulta movimentações, auditoria e relatórios. O cancelamento de um pedido ativo é administrativo.

Verificação de acesso

O papel vem das especializações ligadas ao usuário. Ao acessar um pedido individual como cliente, o backend percorre pedido, sessão e reserva até localizar o dono. Um pedido de outro cliente recebe 404. Uma operação fora do papel permitido recebe 403.

Exemplo. Um cozinheiro pode avançar um ticket para em preparo. Se tentar cancelá-lo com o mesmo método PATCH, a mudança falha porque o cancelamento exige o papel de administrador.

Arquivos consultados: backend/app/controllers/http.py e api_controller.py · FrontEnd/README.md

2.3 Modelo de dados e ciclo do atendimento

O modelo reúne 17 tabelas. As chaves de relacionamento registram a sequência comercial e as tabelas de estoque registram os insumos usados. O dicionário final descreve os campos, tipos e vínculos de cada tabela.

Identidade e especialização

usuario concentra os dados de acesso. cliente, cozinheiro e administrador detalham o perfil. sessao_autenticacao registra a validade do login. limite_acesso e evento_auditoria registram, respectivamente, tentativas de acesso e eventos da API.

Cadeia operacional

Uma bancada pode ter várias reservas. Cada reserva liga uma bancada a um cliente e pode originar uma sessão. Uma sessão pode reunir vários pedidos. Cada pedido guarda o status operacional e os valores oficiais. item_pedido guarda quantidade, subtotal e dados históricos do prato.

Catálogo e estoque

ficha_tecnica associa prato a ingrediente e define o consumo por porção. ingrediente_substituicao associa um ingrediente original a um substituto direto, com prioridade e fator. movimentacao_estoque vincula a operação ao saldo alterado e pode guardar o pedido e o ingrediente original.

Consulta histórica

O preço e o nome do prato podem mudar no catálogo. Os valores registrados no item continuam sendo os da confirmação, o que permite explicar o total de uma venda antiga. O ingrediente original da ficha e o ingrediente efetivamente debitado podem ser diferentes, por isso a movimentação registra ambos quando há substituição.

Chaves compostas distinguem as combinações prato/ingrediente e original/substituto. A sessão de atendimento tem vínculo único com a reserva.

Arquivos consultados: backend/database/01_modelo_fisico.sql · backend/app/models/ · Documentação/Dicionario_final.pdf

2.4 Bancadas, cardápio e cadastros

Antes do primeiro atendimento, a operação precisa de bancada ativa, catálogo e insumos cadastrados. A reserva consulta a bancada, enquanto o checkout consulta os pratos e as receitas vinculadas.

Bancadas

O cadastro define nome, tarifa por hora e disponibilidade cadastral. A busca por intervalo combina a situação da bancada com a existência de reservas sobrepostas. A validação é repetida na gravação, pois outra pessoa pode reservar entre a consulta de disponibilidade e o clique de confirmação.

Pratos e ingredientes

O prato tem preço oficial e indicador de atividade. Inativar o prato impede novos pedidos e preserva seu vínculo com o histórico. Os ingredientes registram unidade e saldo. As correções de saldo integram o histórico de movimentações. Ingredientes com movimentações têm exclusão protegida na API.

Receita e configuração de alternativas

O administrador cadastra a quantidade de cada ingrediente para uma unidade do prato. Uma alteração da ficha muda o cálculo de pedidos futuros. As regras de substituição informam qual alternativa pode atender ao original, em qual ordem deve ser considerada e como converter a quantidade.

Exemplo. Alterar o preço de um prato de R$ 30 para R$ 32,50 muda a confirmação de novas compras. O histórico dos itens já confirmados mantém o preço unitário gravado à época.

Arquivos consultados: backend/app/controllers/api_controller.py · backend/app/controllers/estoque_controller.py · backend/database/01_modelo_fisico.sql

3 RESERVAS E ATENDIMENTO

3.1 Reserva, conflito de horário e tarifa

Uma reserva confirmada associa cliente, bancada, início, fim e valor. O servidor verifica a elegibilidade da bancada e compara o intervalo solicitado com as reservas que ocupam o mesmo espaço.

Regra do intervalo

O início deve estar no futuro e o fim deve ser posterior ao início. Há conflito quando o início existente é anterior ao fim solicitado e o fim existente é posterior ao início solicitado. O teste considera reservas confirmadas e em andamento. Os extremos contíguos são permitidos pela comparação estrita.

Concorrência

O servidor bloqueia a linha da bancada antes da consulta de conflito. No MySQL, confirmações concorrentes que disputam a mesma bancada passam pelo mesmo bloqueio. A segunda requisição reavalia a disponibilidade depois da primeira. No reagendamento, a busca exclui o próprio registro.

Tarifa e manutenção

O valor total multiplica a tarifa por hora pela duração em segundos dividida por 3.600. O cálculo usa Decimal e quantiza para centavos. Só uma reserva confirmada pode passar por alteração ou cancelamento no fluxo de manutenção da reserva.

Bancada a R$ 40/h. Um intervalo de 90 minutos custa R$ 60. Se já existe reserva das 14h às 15h, a solicitação das 14h30 às 15h30 gera 409. Começar às 15h atende a regra de contiguidade.

Figura 1 – Trecho de código: reserva, conflito de horário e tarifa

operacao_controller.py / linhas 54-63

```
54      end = date(data.get('fim', record.fim if record else None), 'fim')
55      if end <= start or start < now():
56          fail('Informe um período futuro com fim posterior ao início.')
57      query = select(m.Reserva).where(m.Reserva.id_bancada == bid,
58          m.Reserva.status.in_(('confirmada', 'em_andamento')),
59          m.Reserva.inicio < end, m.Reserva.fim > start)
60      if record:
61          query = query.where(m.Reserva.id_reserva != record.id_reserva)
62      if db.session.scalar(query.with_for_update()):
63          fail('Esta bancada já está reservada neste período.', 409)
```

Fonte: código do projeto Sabor e Clic (2026).

Arquivos consultados: backend/app/controllers/operacao_controller.py · reserve()

3.2 Sessão de atendimento e encerramento

A sessão identifica o atendimento em andamento e serve de referência para os pedidos. Ela é criada por check-in e tem ciclo de vida próprio, vinculado à reserva.

Abertura

POST /api/sessoes recebe id_reserva. A API carrega e bloqueia a reserva, valida o vínculo, exige status confirmada e verifica se o instante atual pertence ao intervalo. O início é inclusivo e o fim é exclusivo. Ao criar a sessão ativa, o servidor muda a reserva para em_andamento.

Atendimento ativo

O código de acesso da sessão é um identificador operacional. A identidade do usuário é verificada pela sessão de autenticação. Se houver atendimentos simultâneos em bancadas distintas, a interface permite escolher o contexto do carrinho. O backend confere de novo a propriedade e o estado da sessão no checkout.

Encerramento

Após o fim da janela, novos pedidos ficam bloqueados. O encerramento exige que todo pedido esteja entregue ou cancelado. A API então grava o horário de encerramento, marca a sessão como encerrada e conclui a reserva. Uma tentativa com pedido ainda em preparo recebe 409.

Numa reserva das 14h às 15h, o cliente pode iniciar às 14h. Às 15h, a janela já terminou. Um pedido aceito antes disso pode seguir pelo fluxo da cozinha até a entrega.

Figura 2 – Trecho de código: sessão de atendimento e encerramento

operacao_controller.py / linhas 83–90

```
83      reservation = owned(get(m.Reserva, integer(data.get('id_reserva'),
         'id_reserva'), lock=True))
84      if reservation.status != 'confirmada':
85          fail('A reserva não está confirmada.', 409)
86      if now() >= reservation.fim:
87          fail('O período desta reserva já terminou.', 409)
88      if now() < reservation.inicio:
89          fail('Aguarde o horário de início da reserva.', 409)
90      if db.session.scalar(select(m.Sessao).where(m.Sessao.id_reserva ==
         reservation.id_reserva)):
```

Fonte: código do projeto Sabor e Clic (2026).

Arquivos consultados: backend/app/controllers/operacao_controller.py · start_session() e close_session()

3.3 Datas, dinheiro e validação de entradas

Os dados de tela passam por conversão e validação no servidor antes de orientar reservas, compras ou movimentações. A normalização permite comparar horários e quantidades de forma consistente.

Datas e fusos

O backend lê datas ISO 8601. Valores sem offset recebem a referência UTC−03:00 usada para São Paulo. Os horários são convertidos a UTC para persistência e retornam no JSON com offset UTC. A interface apresenta os horários de São Paulo. Assim, 14h com offset −03:00 representa o mesmo instante que 17h UTC.

Preços e quantidades

O carrinho calcula a estimativa em centavos inteiros. Na API, valores decimais passam por validação e quantização. O cálculo do preço oficial usa os registros consultados no checkout. Estoque usa três casas decimais e o fator de conversão cadastrado tem quatro casas no esquema.

Entradas textuais e numéricas

O corpo JSON precisa ser um objeto. Identificadores e quantidades usam inteiros dentro dos limites definidos. O checkout limita a lista a 100 entradas e a quantidade agregada por prato a 1.000. Observações têm limite de 500 caracteres. O servidor preserva os caracteres da senha ao verificá-la.

Exemplo. O preço unitário de R$ 32,50 corresponde a 3.250 centavos no navegador. Duas unidades exibem estimativa de R$ 65,00. O total oficial é calculado com o preço consultado na confirmação.

Arquivos consultados: backend/app/controllers/http.py · operacao_controller.py · FrontEnd/assets/domain.js

3.4 Carrinho, estado local e envio

As entidades do frontend representam os dados recebidos e encapsulam as regras necessárias para a interação. A persistência do atendimento ocorre por meio da API.

Entidades e valores

Entidade fornece um identificador validado. Usuario, Prato e ItemPedido estendem essa base. Prato converte o preço em centavos e ItemPedido calcula o subtotal. Carrinho mantém um mapa por identificador, soma unidades do mesmo prato e oferece operações para remover ou alterar quantidade.

Preparação do payload

Pedido usa o identificador da sessão selecionada e serializa os itens com id_prato e quantidade. Endereço e observações são opcionais. A estimativa exibida é informativa. A escolha da sessão e a lista de itens são verificadas no backend, inclusive quando o cliente mantém a página aberta enquanto a situação do atendimento muda.

Envio e resposta

A tela desabilita o botão enquanto a requisição está em curso. Antes de enviar, guarda a assinatura e a chave da operação pendente. Após sucesso, remove o estado pendente, limpa o carrinho e abre a área de pedidos. Se o envio falhar, mostra a mensagem e habilita uma nova tentativa.

Exemplo de corpo JSON. {"id_sessao":7,"itens":[{"id_prato":3,"quantidade":2}],"observacoes":"Separar os talheres"}.

Figura 3 – Trecho de código: carrinho, estado local e envio

views.js / linhas 190–196

```
190     const storageKey='pending-order:'+currentUser.id_usuario;
191     let pending;
192     try{pending=JSON.parse(localStorage.getItem(storageKey)||'null');}
        catch{pending=null;}
193     const
        key=pending?.assinatura===order.assinatura?pending.chave:order.cha
        ve;
194     localStorage.setItem(storageKey,JSON.stringify({assinatura:order.a
        ssinatura,chave:key}));
195     await api('/pedidos',{method:'POST',headers:{'Idempotency-
        Key':key},body:order.toPayload()});
196     localStorage.removeItem(storageKey);
```

Fonte: código do projeto Sabor e Clic (2026).

Arquivos consultados: FrontEnd/assets/domain.js · views.js · api.js

4 PEDIDOS, ESTOQUE E COZINHA

4.1 Checkout e gravação do pedido

O checkout transforma a intenção do cliente em um pedido recebido. A confirmação só ocorre depois de validar a sessão, calcular os valores e aplicar as movimentações de estoque.

Preparação

A API bloqueia a sessão de atendimento, valida a chave de idempotência, consolida as quantidades por prato e normaliza os campos opcionais. A pesquisa por pedido anterior ocorre antes da validação da janela para permitir recuperar uma confirmação já registrada.

Preço e catálogo

Para uma operação nova, a sessão precisa estar ativa, a reserva em andamento e o horário dentro do período. O controller consulta os pratos em ordem de identificador e bloqueia as linhas consultadas. Rejeita prato inativo, calcula subtotais pelo preço oficial e preserva o nome e o preço unitário no item.

Gravação e resposta

O backend calcula a alocação dos ingredientes, adiciona o pedido e faz flush para obter seu ID. Grava os itens, aplica o consumo e faz um único commit. A resposta inclui itens, consumos e a bancada. Se alguma etapa falhar antes do commit, os handlers desfazem a transação.

Fluxo de persistência. Calcular alocação → inserir pedido → inserir itens → registrar consumo → confirmar transação. Flush torna os dados disponíveis dentro da transação e o commit torna o conjunto persistente.

Figura 4 – Trecho de código: checkout e gravação do pedido

operacao_controller.py / linhas 170–177

```
170      db.session.add(order)
171      db.session.flush()
172      for dish, quantity, subtotal in prepared:
173          db.session.add(m.ItemPedido(id_pedido=order.id_pedido,
         id_prato=dish.id_prato,
174                quantidade=quantity, preco_unitario=dish.preco,
         subtotal=subtotal,nome_prato=dish.nome))
175      db.session.flush()
176      consume(order.id_pedido,allocations)
177      db.session.commit()
```

Fonte: código do projeto Sabor e Clic (2026).

Arquivos consultados: backend/app/controllers/operacao_controller.py · checkout() e order_json()

4.2 Reenvio, chave de idempotência e concorrência

Um pedido pode chegar ao servidor e ser confirmado mesmo se a resposta se perder. A chave de idempotência permite descobrir o resultado da tentativa anterior sem criar outro pedido.

Chave do navegador

O frontend gera uma chave, vincula a chave à assinatura do payload e guarda o par antes do envio. Enquanto o conteúdo permanecer igual, uma tentativa após falha usa a chave pendente. Quando o cliente modifica a operação, o navegador gera uma chave para o novo conteúdo.

Identidade no servidor

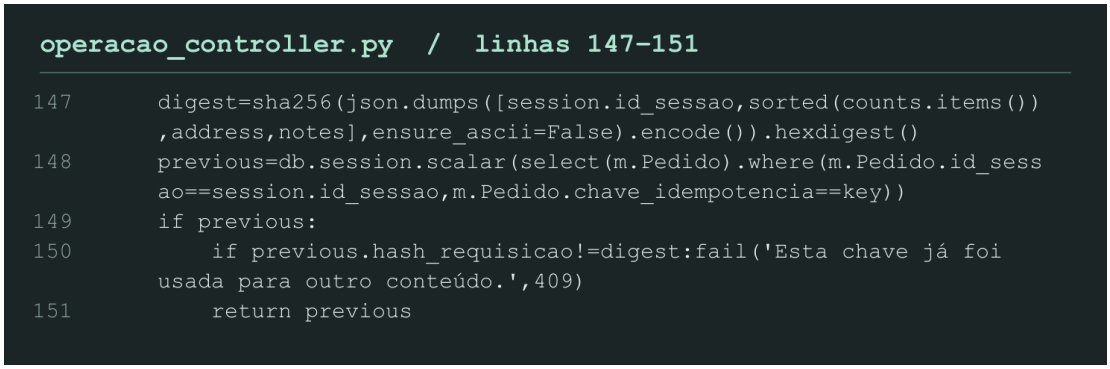
O cabeçalho aceita de 16 a 64 caracteres alfanuméricos, hífen ou sublinhado. O backend procura a chave dentro da sessão de atendimento. Compara o hash da sessão, da lista ordenada de quantidades agregadas, do endereço e das observações. Alterar a ordem das entradas sem mudar as quantidades mantém o conteúdo equivalente para esse cálculo.

Resultados

Se chave e hash coincidem, a API retorna o mesmo pedido e os consumos existentes. Se o hash mudou, a resposta é 409. Duas requisições simultâneas para a mesma sessão passam pelo bloqueio da sessão antes de consultar a chave. A restrição de unicidade da chave na sessão reforça a proteção no banco.

Se o pedido nº 42 foi confirmado e a rede caiu antes da resposta, o reenvio retorna novamente o pedido nº 42. O handler usa HTTP 201 também nesse retorno.

Figura 5 – Trecho de código: reenvio, chave de idempotência e concorrência



Fonte: código do projeto Sabor e Clic (2026).

Arquivos consultados: FrontEnd/assets/views.js · backend/app/controllers/operacao_controller.py · api_controller.py

4.3 Ficha técnica e agregação por ingrediente

A ficha técnica define a quantidade de cada ingrediente consumida por unidade de prato. A demanda é agregada por ingrediente para considerar todo o pedido antes de consultar a viabilidade de preparo.

Cálculo da necessidade

Para cada prato, o backend multiplica a quantidade da ficha pelo número de unidades e soma o resultado à demanda do ingrediente. Duas porções com 0,150 kg consomem 0,300 kg. Outro prato com 0,100 kg do mesmo insumo leva o total a 0,400 kg.

Prioridade dos originais

O backend identifica ingredientes originais e substitutos possíveis, consulta as linhas envolvidas com bloqueio e monta um mapa dos saldos disponíveis. Na primeira passagem, reserva os originais que conseguem cobrir sua própria necessidade. A segunda passagem procura alternativas para os originais que ficaram pendentes.

Interação entre receitas

O mapa de saldos é compartilhado por todas as alocações. Quando uma quantidade é reservada, ela deixa de estar disponível para as próximas decisões daquele pedido. Isso impede que o mesmo saldo seja contado duas vezes, inclusive quando um ingrediente é original em um prato e candidato a substituto em outro.

Exemplo. Se o ingrediente B é necessário para sua própria receita e também pode substituir A, a quantidade destinada à receita de B é considerada antes de usar seu saldo como alternativa para A.

Arquivos consultados: backend/app/controllers/estoque_controller.py · allocate()

4.4 Substituição, fator e arredondamento

Uma alternativa autorizada pode cobrir a falta de um ingrediente original. A escolha respeita a configuração administrativa e o saldo que ainda pode ser usado.

Seleção direta

Para cada original pendente, o backend percorre os substitutos diretos em ordem de prioridade e usa o identificador do substituto para desempate. A alternativa precisa atender à demanda inteira após a conversão. O algoritmo avalia uma alternativa por vez, sem combinar saldos parciais ou percorrer uma cadeia de substituições.

Conversão e saldo

A quantidade necessária do substituto é a quantidade do original multiplicada pelo fator de conversão. O controller arredonda para cima na terceira casa decimal. Se um fator exige 0,5001 unidade, o débito previsto é 0,501. A unidade do cadastro e o fator devem representar a medida usada na receita.

Registro do consumo

Quando existe substituição, a movimentação identifica o ingrediente efetivamente debitado e o original. Isso permite explicar por que o saldo de um ingrediente diferente mudou. Se nenhuma alternativa cobrir a necessidade, o checkout retorna 409 e a transação é desfeita.

Para 0,400 kg de demanda e fator 1,25, a alternativa precisa de 0,500 kg. Com saldo de 0,499 kg, o sistema procura o próximo substituto elegível.

Figura 6 – Trecho de código: substituição, fator e arredondamento

estoque_controller.py / linhas 45–53

```
45          for alternative in alternatives:
46              if alternative.id_ingrediente_original!=original:continue
47              amount=(required*alternative.fator_conversao).quantize(Dec
        imal('.001'),rounding=ROUND_CEILING)
48              substitute=alternative.id_ingrediente_substituto
49              if available.get(substitute,0)>=amount:
50                  available[substitute]-
        =amount;result.append((substitute,original,amount));break
51          else:fail(f'Estoque insuficiente para o ingrediente
        #{original} e seus substitutos.',409)
52      return result
53
```

Fonte: código do projeto Sabor e Clic (2026).

Arquivos consultados: backend/app/controllers/estoque_controller.py · allocate() e consume()

4.5 Movimentações, saldo e estorno

A tabela de movimentações registra a história do estoque. O saldo em ingrediente é atualizado junto ao lançamento, permitindo consultas rápidas e rastreabilidade da alteração.

Débito e integridade

Ao confirmar o pedido, consume() grava uma movimentação negativa para a alocação correspondente. A atualização SQL exige que saldo atual mais quantidade seja maior ou igual a zero. Quando nenhuma linha pode ser atualizada, o backend responde com conflito e desfaz a transação. As chaves de evento identificam consumos e estornos.

Cancelamento antes do preparo

Se um administrador cancela em recebido, restock() lê os consumos do pedido e lança a quantidade inversa para cada ingrediente efetivamente debitado. O cancelamento e o estorno são confirmados juntos. O débito continua no histórico e fica acompanhado do crédito.

Cancelamento depois do início

Quando o preparo já começou, cancelar encerra o pedido e conserva o débito. Caso exista diferença física a registrar, a correção é uma nova movimentação. As movimentações anteriores permanecem como evidência de como o saldo chegou ao valor atual.

Se foram debitados 0,500 kg do substituto B e o cancelamento ocorrer em recebido, o estorno credita 0,500 kg de B. O original A continua identificado como referência da receita.

Arquivos consultados: backend/app/controllers/estoque_controller.py · movement(), consume() e restock() · operacao_controller.py

4.6 KDS, estimativa e estados do preparo

O Kitchen Display System apresenta a fila de pedidos em operação. Os estados informam o estágio do trabalho e orientam a próxima ação permitida.

Entrada na fila

O checkout grava o pedido como recebido. O KDS consulta pedidos ativos, com ordenação crescente por identificador na fila operacional, e mostra dados como bancada, itens e observações. A consulta é paginada. A tela se atualiza periodicamente.

Sequência de execução

A cozinha registra tempo_estimado em minutos. O status avança de recebido para em_preparo, depois para pronto e entregue. O backend valida a transição e grava os horários operacionais. Tentar ir diretamente de recebido para pronto retorna 409.

Saída e acompanhamento

Entregue e cancelado são estados finais. A entrega pode ser conferida pelo cliente e a cozinha pode localizar o registro no histórico. O administrador pode cancelar um pedido ainda ativo. A transição é avaliada sob bloqueio do pedido, de modo que a regra é aplicada ao estado corrente.

No ticket da bancada Toscana usado na demonstração, o pedido está em preparo com estimativa de oito minutos. A ação exibida é Marcar pronto. A captura completa aparece no apêndice.

Arquivos consultados: backend/app/controllers/operacao_controller.py · orders() e update_order() · FrontEnd/cozinha_kds.html

5 SEGURANÇA E ACESSO À API

5.1 Login, cookie e sessão revogável

O login estabelece a identidade usada nas requisições protegidas. O servidor mantém o registro de validade para encerrar o acesso por logout, expiração ou mudança de senha.

Autenticação

A API verifica a senha com o hash armazenado. Em caso de sucesso, cria um identificador aleatório e guarda sua impressão no banco com usuário, versão da senha, criação, último acesso e expiração. O cookie Flask carrega o estado de sessão. A configuração usa HttpOnly e SameSite e prevê Secure em HTTPS.

Validade e revogação

A validade absoluta é de oito horas. A configuração padrão encerra a sessão após mais de 30 minutos de inatividade. A verificação compara a versão da senha e confirma se a sessão continua ligada à conta ativa. Logout marca a sessão persistida como revogada e limpa o estado no navegador.

Limite de tentativas

A aplicação usa contadores por IP e por conta. As chaves dos contadores são registradas em hash. Ao atingir o limite dentro da janela configurada, a API responde com 429 e informa Retry-After. Um login bem-sucedido zera o contador da identidade.

O cliente pode estar com reserva em andamento e perder a sessão de login por inatividade. Ao entrar novamente, recupera o acesso ao atendimento que continua vinculado à conta.

Arquivos consultados: backend/app/controllers/seguranca_controller.py · backend/app/controllers/http.py · backend/config.py

5.2 CSRF, propriedade e auditoria

As escritas protegidas passam por verificação de origem, identidade, token, papel e vínculo com o recurso. Cada verificação atua em uma etapa do processamento HTTP.

Requisições protegidas

O frontend obtém o token CSRF por login ou pela leitura de /api/me e o guarda em memória. Nas escritas protegidas, o backend compara X-CSRF-Token ao valor da sessão. A aplicação serve frontend e API na mesma origem e aceita origens adicionais somente quando configuradas.

Regras por recurso

permit() limita o acesso pelo papel. owned() percorre os vínculos de reserva, sessão e pedido para verificar a propriedade no acesso do cliente. A condição da interface ajuda a orientar o uso, mas é a API que decide se o registro pode ser lido ou alterado.

Auditoria e resposta

O registro guarda ação HTTP, caminho, usuário quando identificado, código de resposta e correlação. Leituras bem-sucedidas e respostas 5xx ficam fora dessa trilha. A falha de auditoria gera log com correlação. A resposta inclui X-Request-ID para localizar a ocorrência.

Sem autenticação válida, uma escrita protegida recebe 401. Com login válido e token CSRF ausente ou incorreto, recebe 403. Uma transição de pedido incompatível com o estado atual recebe 409.

Figura 7 – Trecho de código: csrf, propriedade e auditoria

api.js / linhas 2–13

```
2  let csrfToken = '';
3  async function api(path, options = {}) {
4    const headers = {...options.headers};
5    if (options.body !== undefined) headers['Content-Type'] =
       'application/json';
6    if (csrfToken) headers['X-CSRF-Token'] = csrfToken;
7    let response;
8    try {
9      response = await fetch((window.SABOR_API_BASE || '/api') + path, {
10       ...options, headers, credentials: 'include',
11       body: options.body === undefined ? undefined :
       JSON.stringify(options.body)
12     });
13   } catch {
```

Fonte: código do projeto Sabor e Clic (2026).

Arquivos consultados: FrontEnd/assets/api.js · backend/app/__init__.py · backend/app/controllers/http.py e seguranca_controller.py

5.3 Listas, filtros e relatórios

As telas operacionais recebem páginas de registros. Os resumos são calculados separadamente no servidor para representar o conjunto completo, mesmo quando há muitos resultados.

Paginação

pagina começa em 1. limite usa 50 por padrão e aceita até 100. A consulta conta as linhas filtradas e depois aplica offset e limite. A resposta anexa pagina, limite e total sob metadados. O frontend usa esses dados para decidir se há outra página.

Filtros

O backend valida nomes de campos e converte o valor ao tipo da coluna. Nos recursos compatíveis, q pesquisa campos textuais como nome ou descrição. Em pedidos, situacao separa ativos de finalizados. O filtro por criação usa o início inclusivo e o fim exclusivo quando os campos são oferecidos.

Resumos e relatórios

O resumo da sessão conta pedidos e soma o consumo financeiro dos pedidos que permanecem válidos, excluindo cancelados da soma. O relatório por período reúne medidas como receitas, ticket médio, tempo de preparo e resultados por bancada. As consultas SQL do repositório complementam a leitura administrativa com exemplos de venda, estoque e ocupação.

Com 73 pedidos em uma consulta e limite 50, a primeira página retorna até 50 registros e a segunda até 23. O total da sessão continua vindo da agregação feita no servidor.

Arquivos consultados: backend/app/controllers/consulta_controller.py · api_controller.py · backend/database/11_queries_relatorios.sql

5.4 API e diagnóstico de respostas

As rotas sob /api expõem operações de leitura e alteração. O método, o identificador e o estado atual determinam como a operação é interpretada. Os exemplos a seguir assumem login válido.

Rotas operacionais

POST /api/reservas cria uma reserva. PATCH /api/reservas/<id> altera dados de uma reserva elegível. POST /api/sessoes abre o atendimento. POST /api/pedidos confirma o checkout. PATCH /api/pedidos/<id> altera estimativa ou status conforme o perfil e a transição. POST /api/sessoes/<id>/encerrar fecha o atendimento.

Erros de entrada e acesso

400 sinaliza entrada inválida, como data fora do formato ou chave de checkout malformada. 401 pede nova autenticação. 403 indica acesso negado, origem recusada ou token CSRF inválido. 404 indica recurso ausente ou fora do vínculo de um cliente.

Conflitos e indisponibilidade

409 indica conflito de estado ou de dados, como sobreposição de reserva, falta de estoque ou salto no preparo. 429 indica limite de tentativas de login. 503 indica falha de persistência capturada pela aplicação. O corpo contém a mensagem de erro e a resposta informa X-Request-ID.

Ao receber 409 no checkout, leia a mensagem. Sessão fora do período, prato indisponível, estoque insuficiente e reutilização da chave com conteúdo alterado compartilham o mesmo status, mas exigem ações diferentes.

Arquivos consultados: backend/app/routes/api_routes.py · backend/app/__init__.py · backend/app/controllers/http.py

6 INSTALAÇÃO E VALIDAÇÃO

6.1 Instalação, migração e recuperação

O ambiente local usa Python, Flask e MySQL. Os scripts do projeto orientam a instalação, e os comandos administrativos preparam o esquema, os dados iniciais e a recuperação.

Configuração e inicialização

Na raiz, configure a conexão no .env conforme o README. Prepare o esquema usando flask --app run init-db a partir de backend, com uma conta autorizada a criar e alterar tabelas. Use create-admin para configurar a primeira conta ou seed-demo em um ambiente de demonstração. A aplicação é iniciada com run.py.

Migração e conta de execução

O esquema final e as alterações de integração estão no SQL versionado. O comando init-db aplica a preparação do banco e preserva os registros existentes. A conta usada pelo Flask no dia a dia precisa de permissões de leitura e escrita de dados. A conta administrativa é usada nas operações de esquema.

Backup e restauração

backup-db exporta um snapshot de dados para o caminho indicado. restore-db exige um esquema já inicializado e vazio. O processo de recuperação revoga as sessões autenticadas restauradas. Trate o snapshot como confidencial porque contém registros do banco.

Depois de migrar um banco existente, reinicie o Flask com as credenciais de runtime. Valide login, reserva, check-in, pedido e KDS no ambiente alvo.

Arquivos consultados: README.md · backend/app/database.py · backend/app/__init__.py · backend/database/

6.2 Validação e roteiro de demonstração

A verificação registrada em 02/11/2026 cobre regras de aplicação, integração HTTP, concorrência no MySQL e operação no navegador. As evidências têm escopos diferentes.

Testes automatizados

O projeto registrou 41 testes Python e 8 JavaScript. Os casos exercitaram revogação de login, limite de tentativas, datas, preço oficial, consumo, estorno e reenvio. A integração HTTP com banco descartável incluiu listas com mais de 50 registros e totais de sessão que permanecem completos.

MySQL e navegação

No MySQL 8.0.46 isolado, as verificações incluíram pedidos disputando estoque, reservas simultâneas, migração repetível e backup/restauração. No navegador foram observados carrinho, pedido, estimativa, avanço do KDS, reagendamento e encerramento. As capturas do apêndice pertencem a esse ambiente de demonstração.

Como reproduzir o percurso

Prepare os três perfis e uma bancada ativa. Teste primeiro uma sobreposição de reserva. Use a fixture com sessão ativa para exercitar o checkout sem depender de aguardar um horário. Envie duas porções, abra o KDS, registre uma estimativa e avance até entregue. Volte ao cliente e encerre a sessão.

Os resultados registrados em 02/11/2026 correspondem aos ambientes descartáveis descritos nesta seção. Naquela data, a validação do banco real após a migração ainda estava pendente.

Arquivos consultados: backend/tests/ · FrontEnd/tests/ · Documentação/ROTEIRO.md

7 CASO DE PONTA A PONTA PELA API

Considere uma bancada ativa a R$ 40 por hora e um prato de R$ 32,50 que exige 0,150 kg de um ingrediente. O cliente e a cozinha usam contas distintas. Os exemplos assumem autenticação e token CSRF válidos nas escritas.

1. Reserva de 90 minutos

POST /api/reservas solicita 14h–15h30 e retorna 201 com o identificador da reserva e tarifa de R$ 60. Uma segunda reserva na mesma bancada de 15h a 16h retorna 409 porque os intervalos se sobrepõem.

2. Abertura da sessão

Às 14h, POST /api/sessoes recebe id_reserva e retorna 201 com id_sessao. Antes do início, a mesma tentativa retorna 409. A reserva passa de confirmada para em_andamento.

3. Confirmação de duas porções

POST /api/pedidos usa id_sessao e Idempotency-Key. O servidor calcula R$ 65, grava o pedido recebido e debita 0,300 kg quando o ingrediente original está disponível. O reenvio do mesmo conteúdo com a mesma chave recupera o mesmo id_pedido.

4. Preparo e entrega

A cozinha consulta GET /api/kds e usa PATCH /api/pedidos/<id> para informar a estimativa e avançar o pedido. O cliente acompanha pelo GET /api/pedidos/<id>. Tentar encerrar a sessão com pedido em preparo retorna 409.

5. Encerramento e conferência

Após a entrega, POST /api/sessoes/<id>/encerrar fecha a sessão e conclui a reserva. O resumo financeiro da sessão mantém a compra, e as movimentações conservam o registro do consumo.

O exemplo resulta em R$ 60 na reserva, R$ 65 no pedido e 0,300 kg de insumo debitado. A reserva e a compra são registros financeiros distintos.

8 CONCLUSÃO

A implementação integra reserva, atendimento e cozinha por meio de vínculos persistidos entre bancada, reserva, sessão e pedido. A API verifica as permissões e valida o estado atual antes de aceitar alterações. O histórico conserva os preços da compra, os horários do preparo e os ingredientes efetivamente debitados.

Os bloqueios de linha e a transação de checkout reduzem conflitos em operações simultâneas. O uso da mesma chave no reenvio recupera a confirmação anterior e evita um novo consumo. A trilha de movimentações permite conferir o efeito do cancelamento sobre o saldo.

Os testes registrados cobrem regras de negócio, integração HTTP e concorrência em MySQL isolado. As evidências do navegador ilustram a operação nos perfis de cliente e cozinha. A seção 6.2 delimita os ambientes usados e a validação após a migração.

REFERÊNCIAS

INSTITUTO FEDERAL DE SÃO PAULO. Biblioteca Francisco Montojos. Guia de orientação à normalização de trabalhos acadêmicos. São Paulo: IFSP, [s. d.]. Disponível em: h ttps://spo.ifsp.edu.br/images/arquivos/documentos_institucionais/Guia_de_Normalizacao.pdf. Acesso em: 12 nov. 2026.

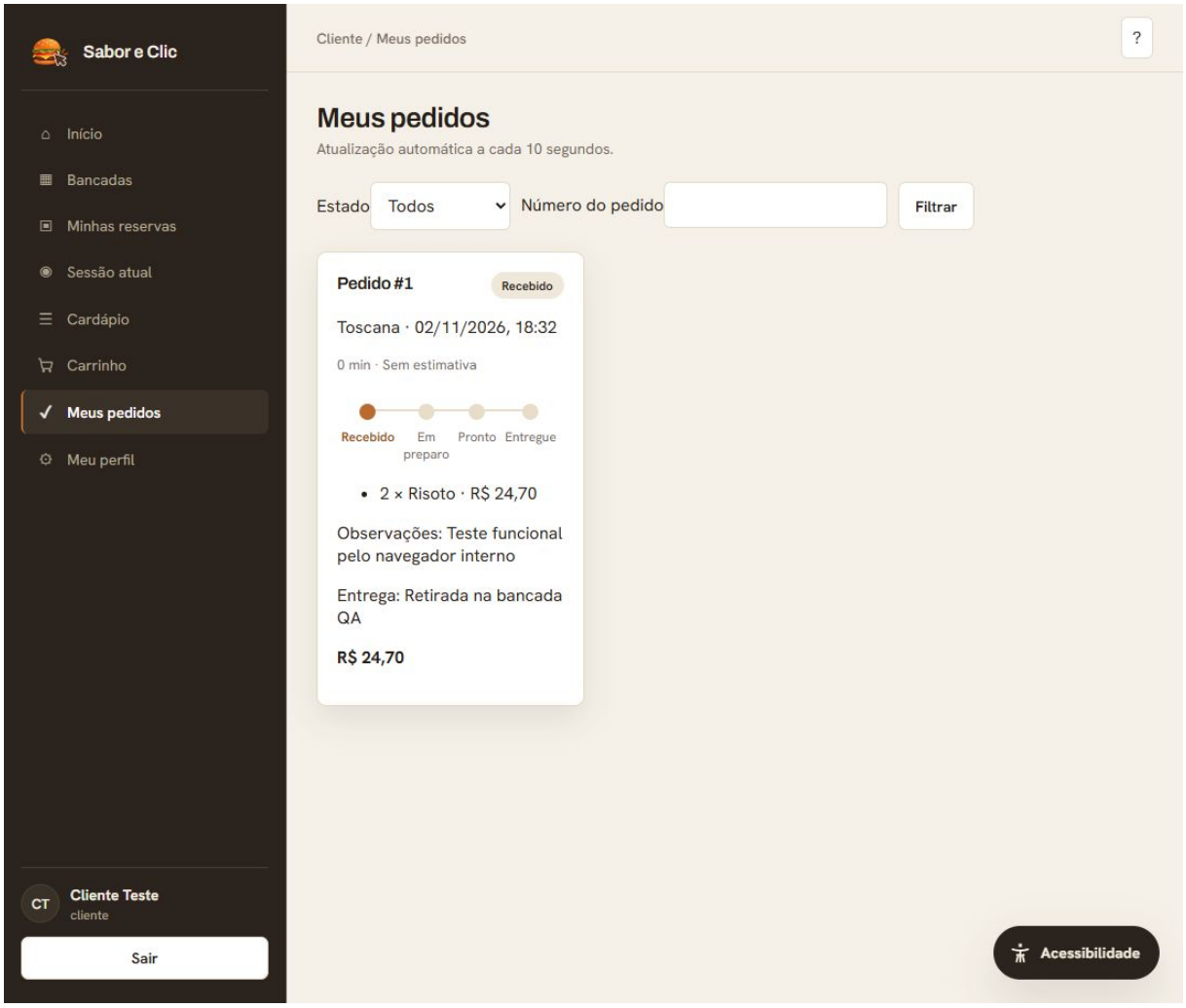
INSTITUTO FEDERAL DE SÃO PAULO. Projeto Integrador Sabor & Clic: sistema de gestão para dark kitchens e espaços gastronômicos. Versão de 31 de agosto de 2026. São Paulo: IFSP, 2026. Documento de orientação do projeto.

SABOR E CLIC. Código-fonte, esquema SQL e documentação do sistema. São Paulo, 2026. Repositório de software.

APÊNDICE A – PEDIDO NO PERFIL DO CLIENTE

A captura mostra o resultado após a confirmação. A tela apresenta os itens, a quantidade, o total, o endereço e as observações do pedido.

Figura 8 – Pedido no perfil do cliente

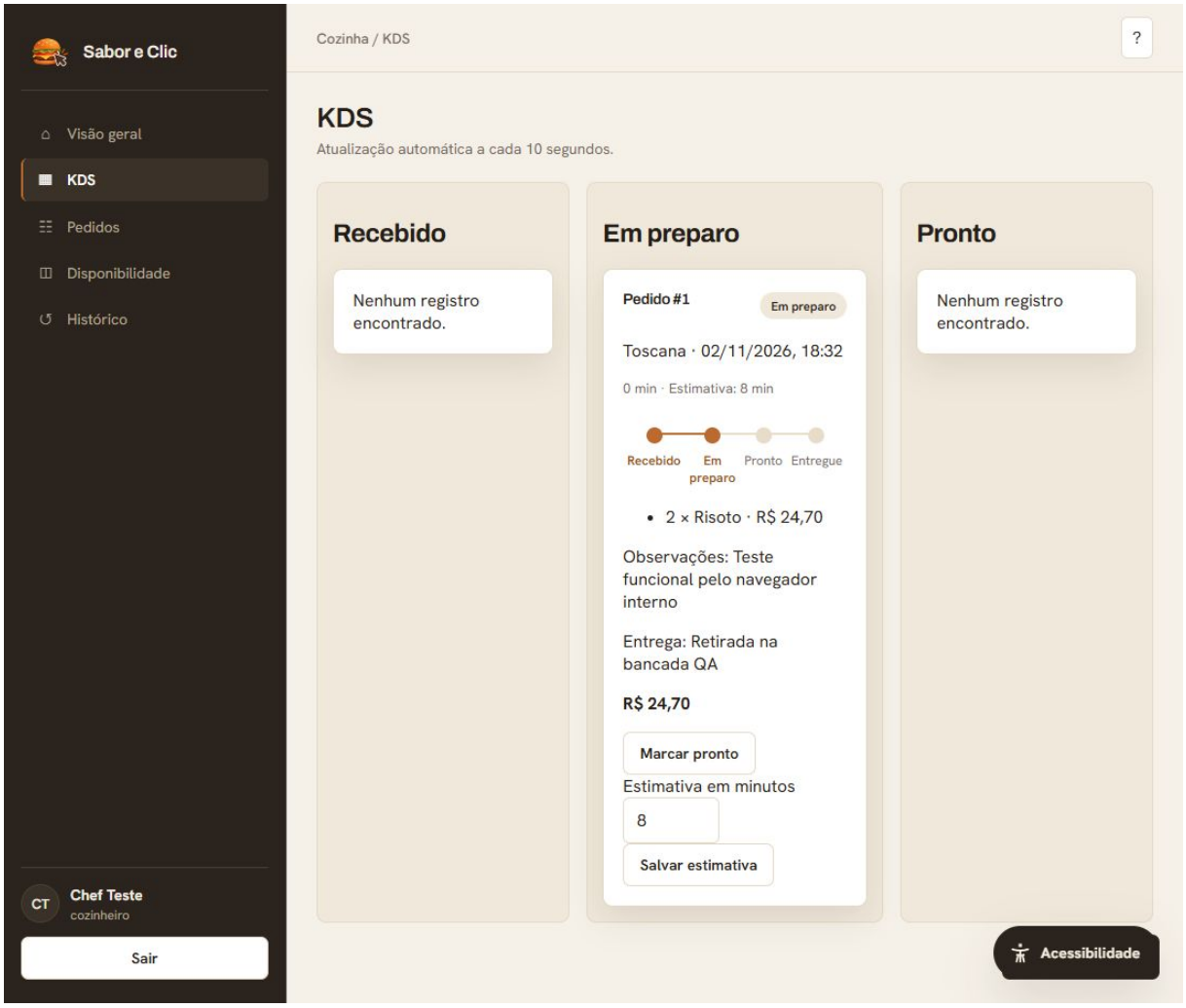


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

APÊNDICE B – PEDIDO EM PREPARO NO KDS

A captura mostra o pedido nº 1 em preparo na bancada Toscana, com estimativa de oito minutos. A ação disponível na tela é Marcar pronto.

Figura 9 – Pedido em preparo no kds



Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

APÊNDICE C – GUIA COMPLETO DAS TELAS

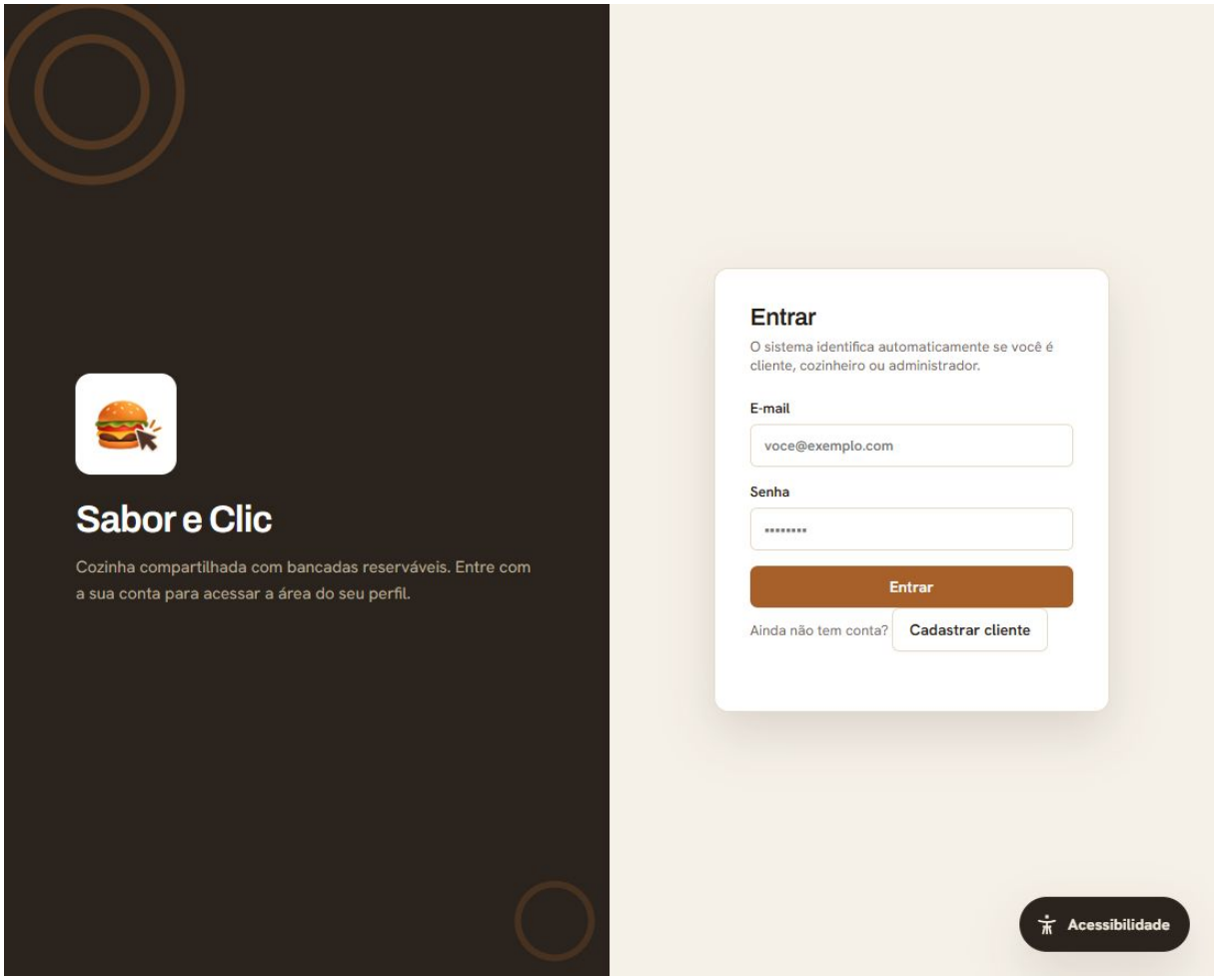
As capturas mostram a aplicação integrada em ambiente de teste. O guia reúne as telas de entrada e dos perfis de cliente, cozinha e administração. Selecione um título para abrir a captura correspondente.

Figura 10 – Login
Figura 11 – Cadastro de cliente
Figura 12 – Cliente · Início
Figura 13 – Cliente · Bancadas
Figura 14 – Cliente · Minhas reservas
Figura 15 – Cliente · Sessão atual
Figura 16 – Cliente · Cardápio
Figura 17 – Cliente · Carrinho
Figura 18 – Cliente · Meus pedidos
Figura 19 – Cliente · Meu perfil
Figura 20 – Cozinha · Visão geral
Figura 21 – Cozinha · KDS
Figura 22 – Cozinha · Pedidos
Figura 23 – Cozinha · Disponibilidade
Figura 24 – Cozinha · Histórico
Figura 25 – Administração · Painel
Figura 26 – Administração · Reservas
Figura 27 – Administração · Bancadas
Figura 28 – Administração · Produtos e pratos
Figura 29 – Administração · Ingredientes
Figura 30 – Administração · Substituições
Figura 31 – Administração · Usuários
Figura 32 – Administração · Relatórios
Figura 33 – Administração · Cadastro de bancada

Login

A entrada autentica os três perfis e direciona cada conta para seu painel.

Figura 10 – Login

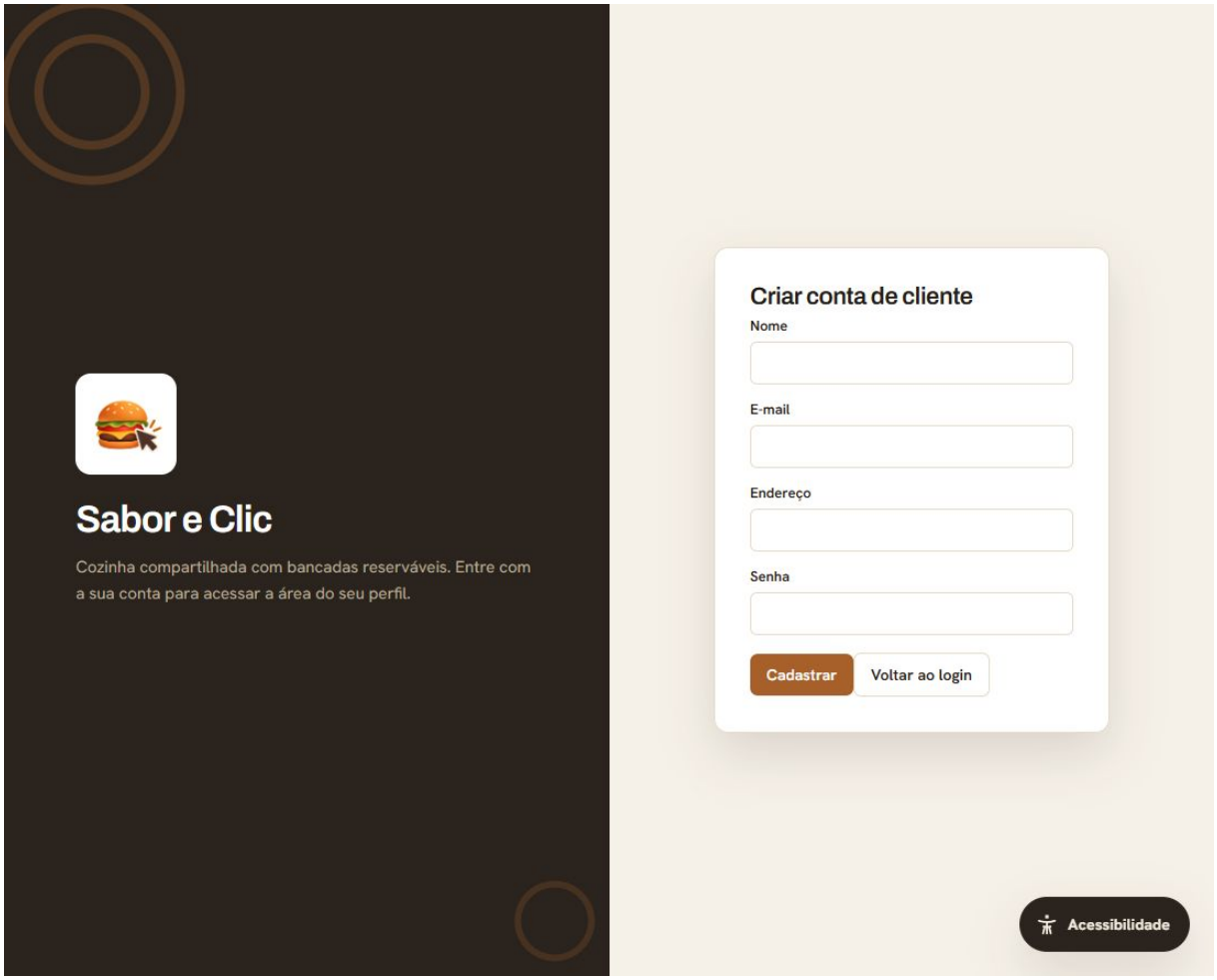


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cadastro de cliente

Cria uma conta de cliente. Contas privilegiadas são cadastradas pela administração.

Figura 11 – Cadastro de cliente

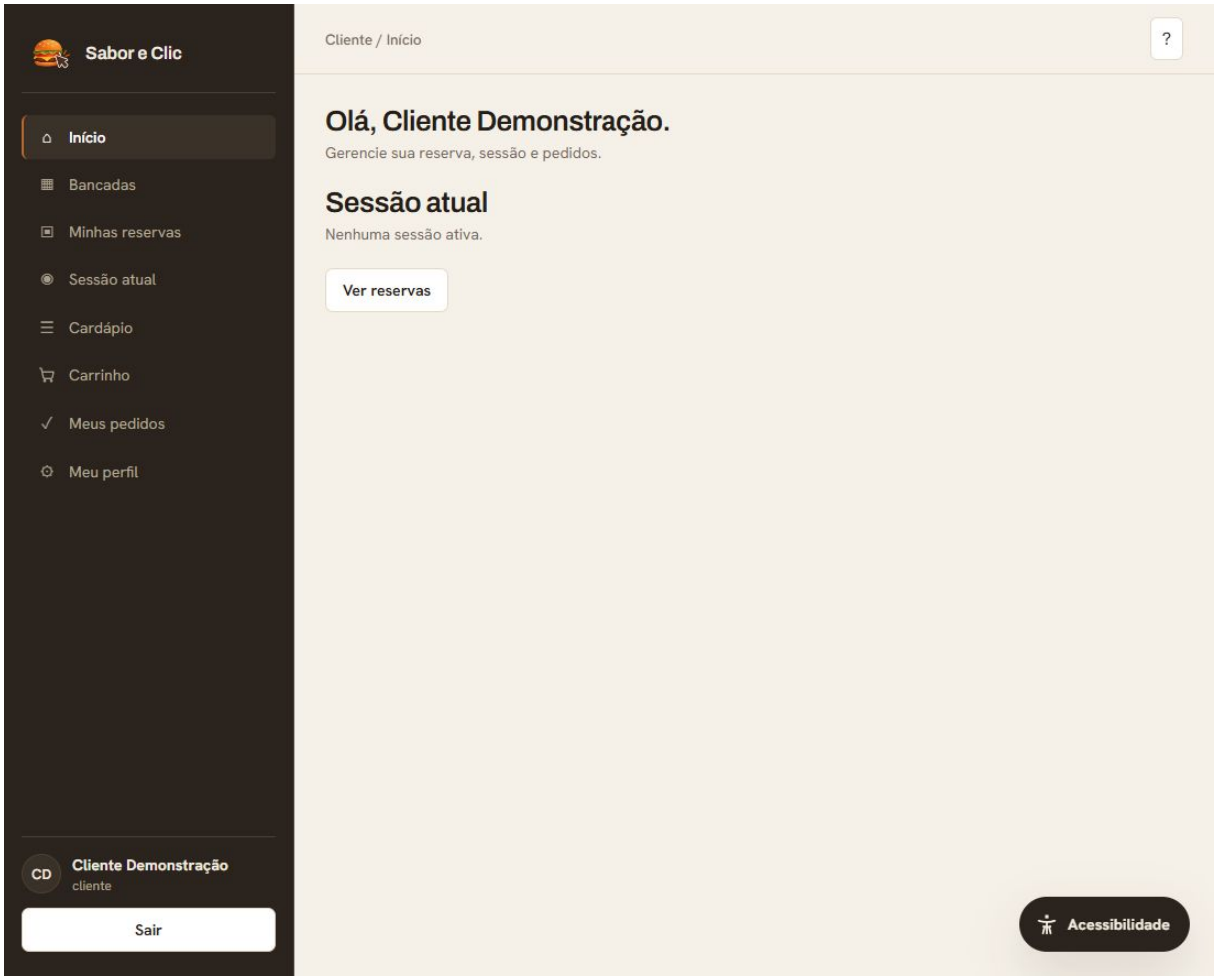


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Início

Ponto de partida para reservas, atendimento e pedidos.

Figura 12 – Cliente · Início

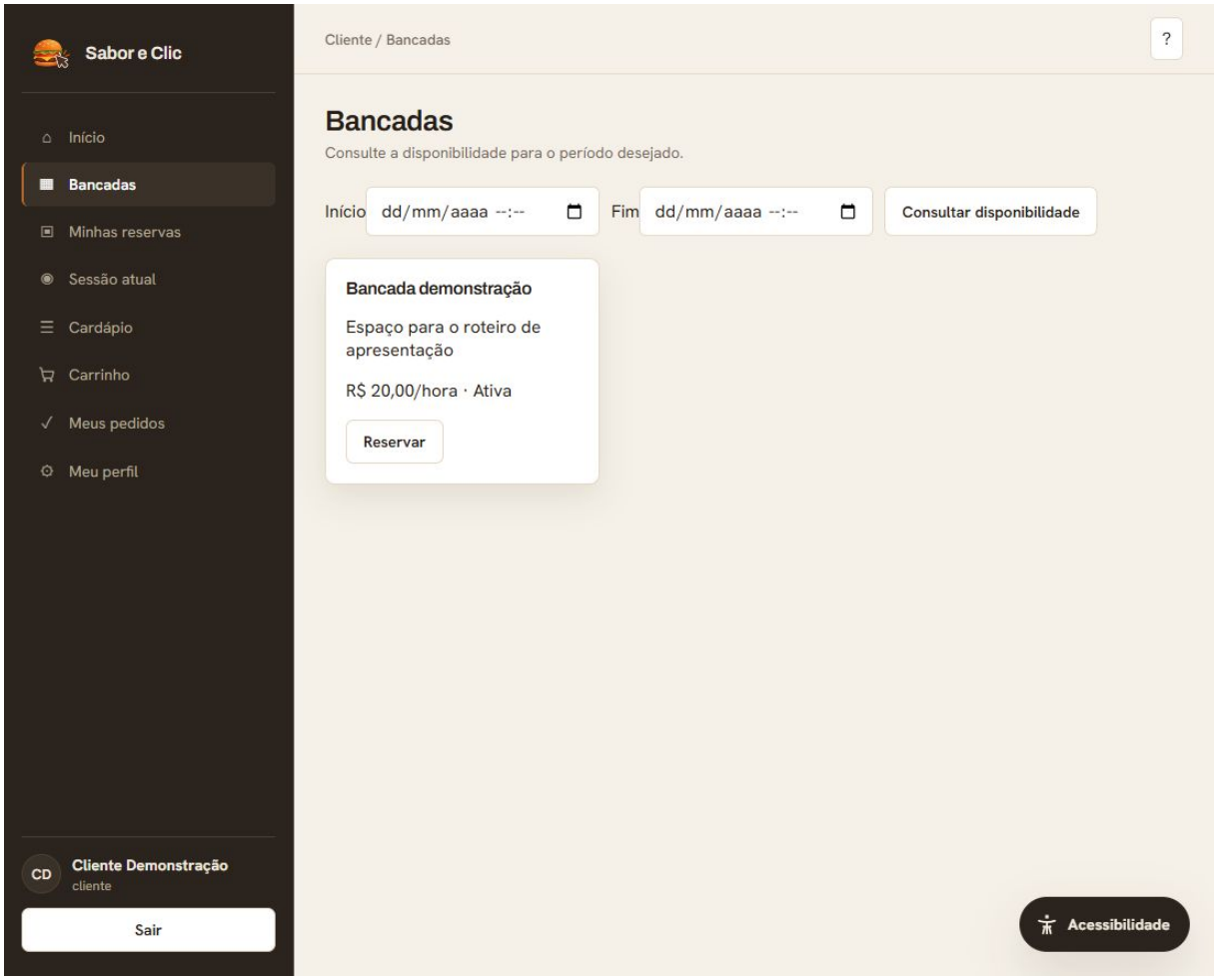


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Bancadas

Consulta disponibilidade por intervalo e apresenta as bancadas e tarifas.

Figura 13 – Cliente · Bancadas

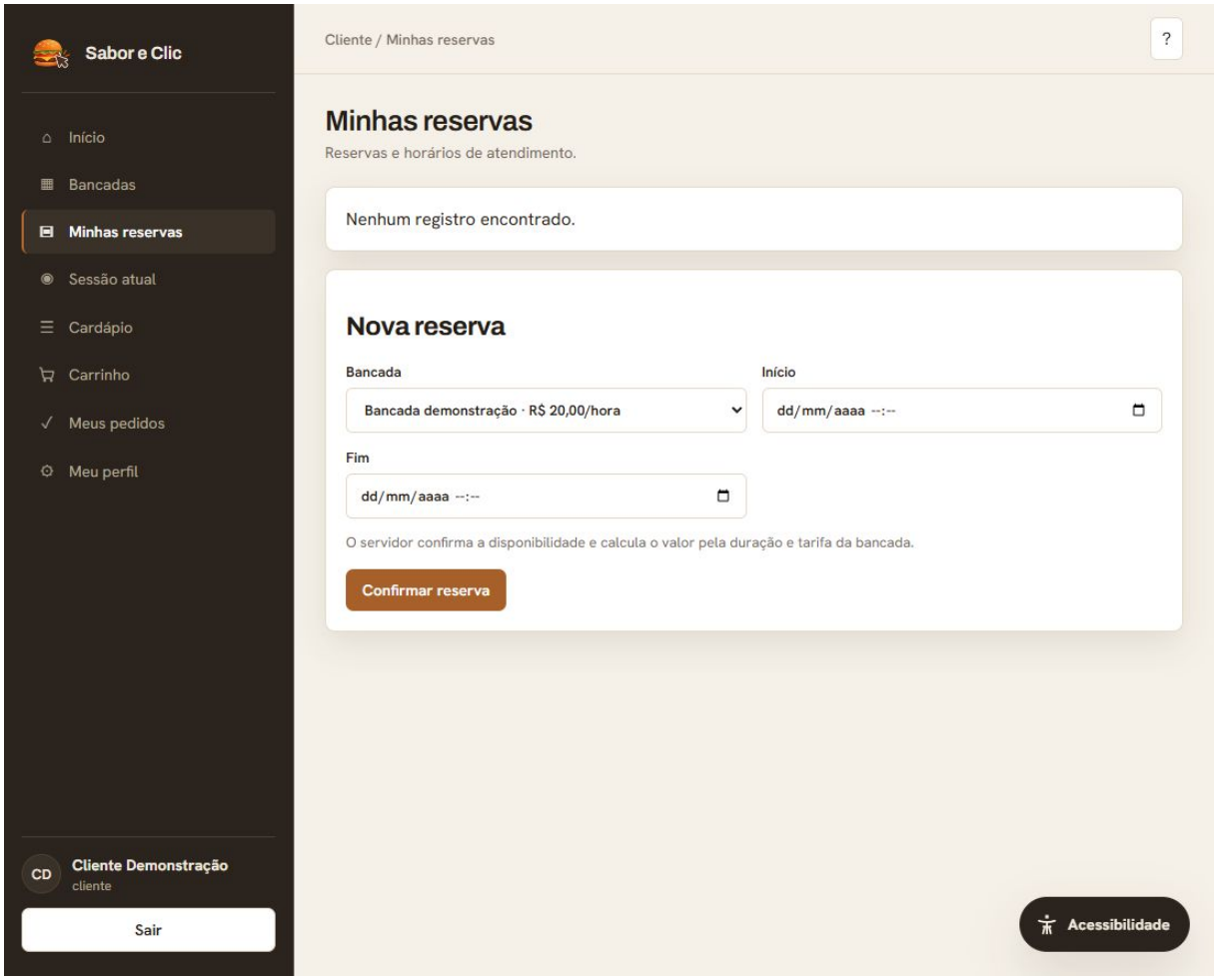


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Minhas reservas

Lista reservas e oferece ações conforme o estado e o horário.

Figura 14 – Cliente · Minhas reservas

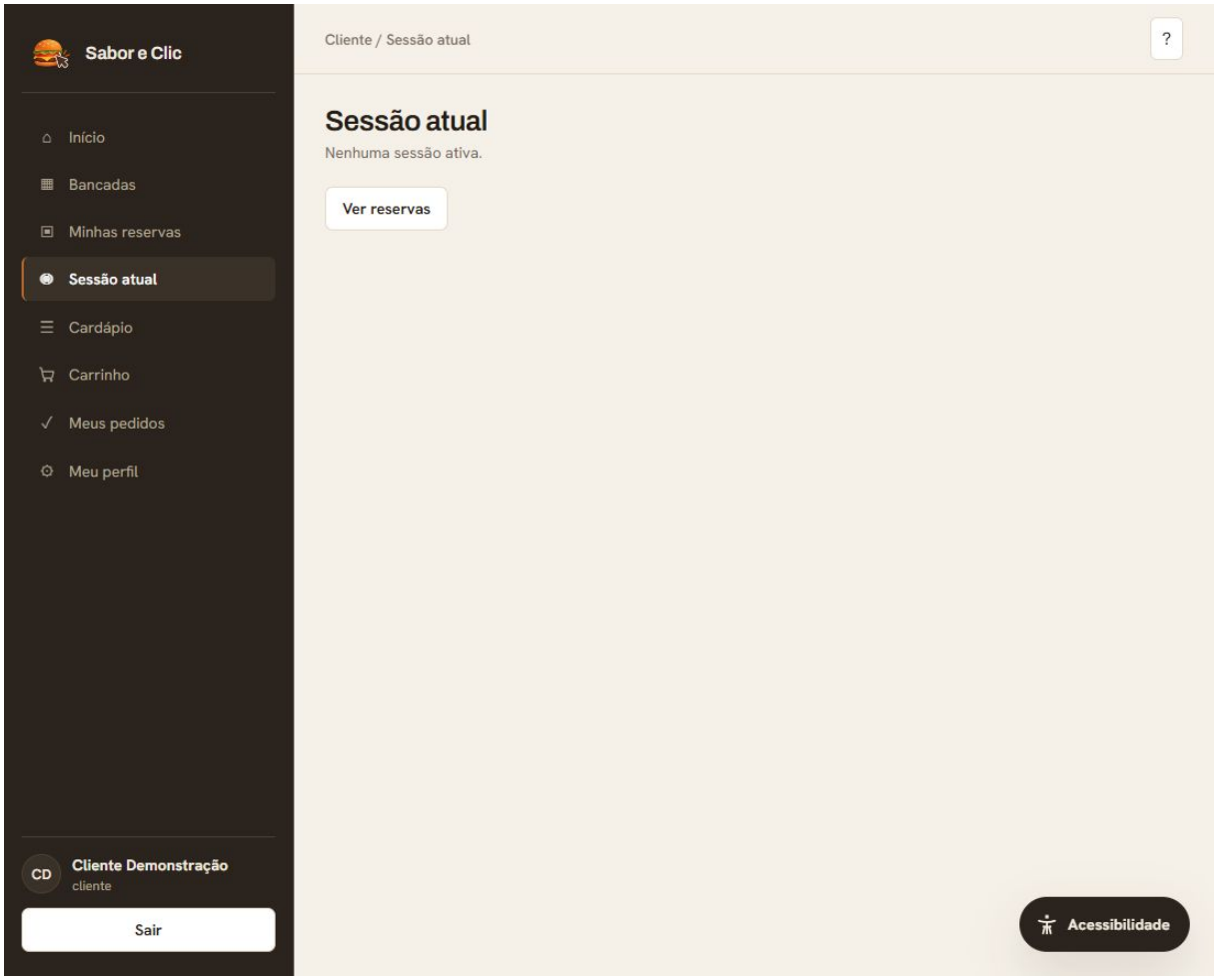


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Sessão atual

Mostra o atendimento selecionado, seus pedidos e o resumo de consumo.

Figura 15 – Cliente · Sessão atual

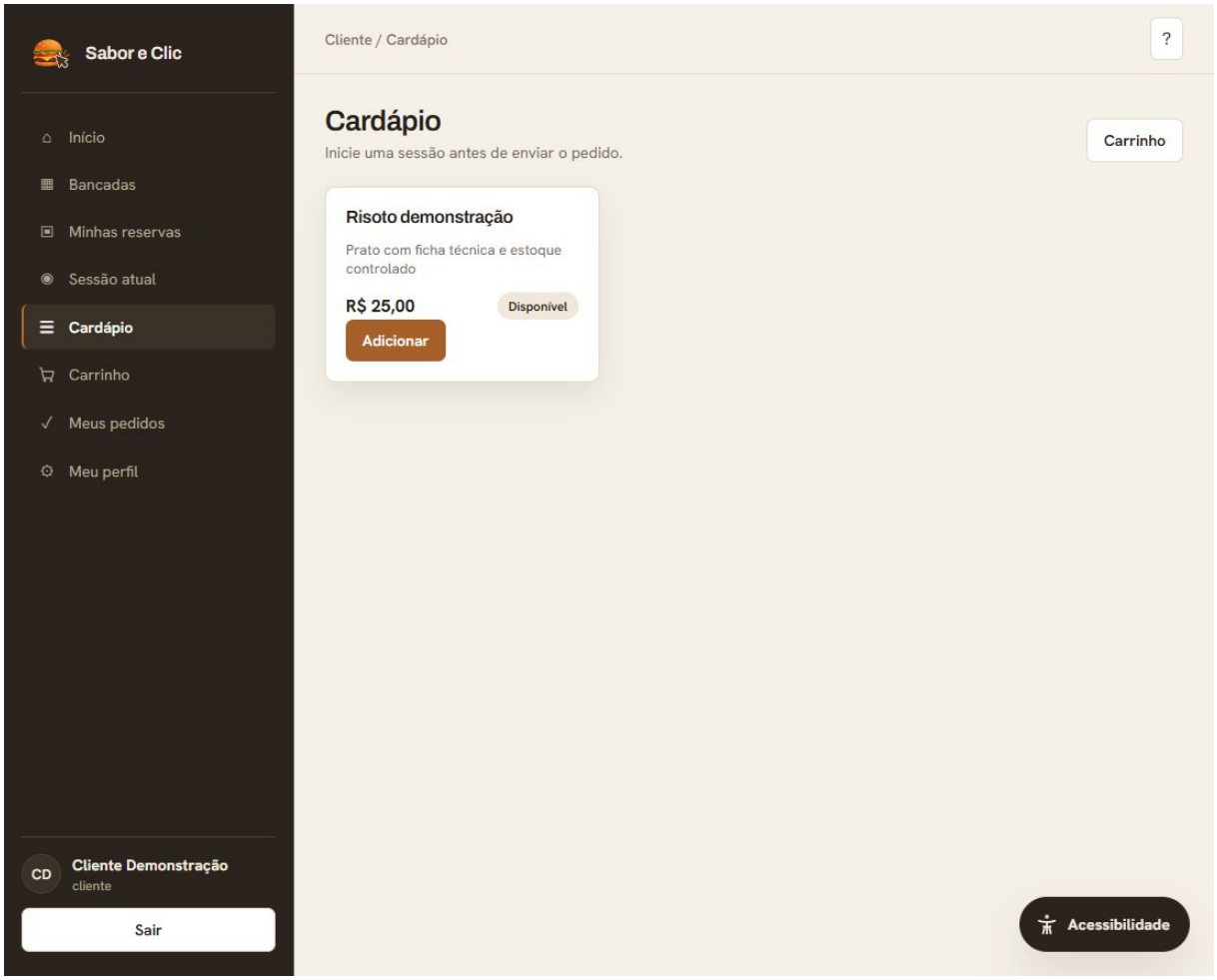


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Cardápio

Permite escolher a sessão e adicionar pratos disponíveis ao carrinho.

Figura 16 – Cliente · Cardápio

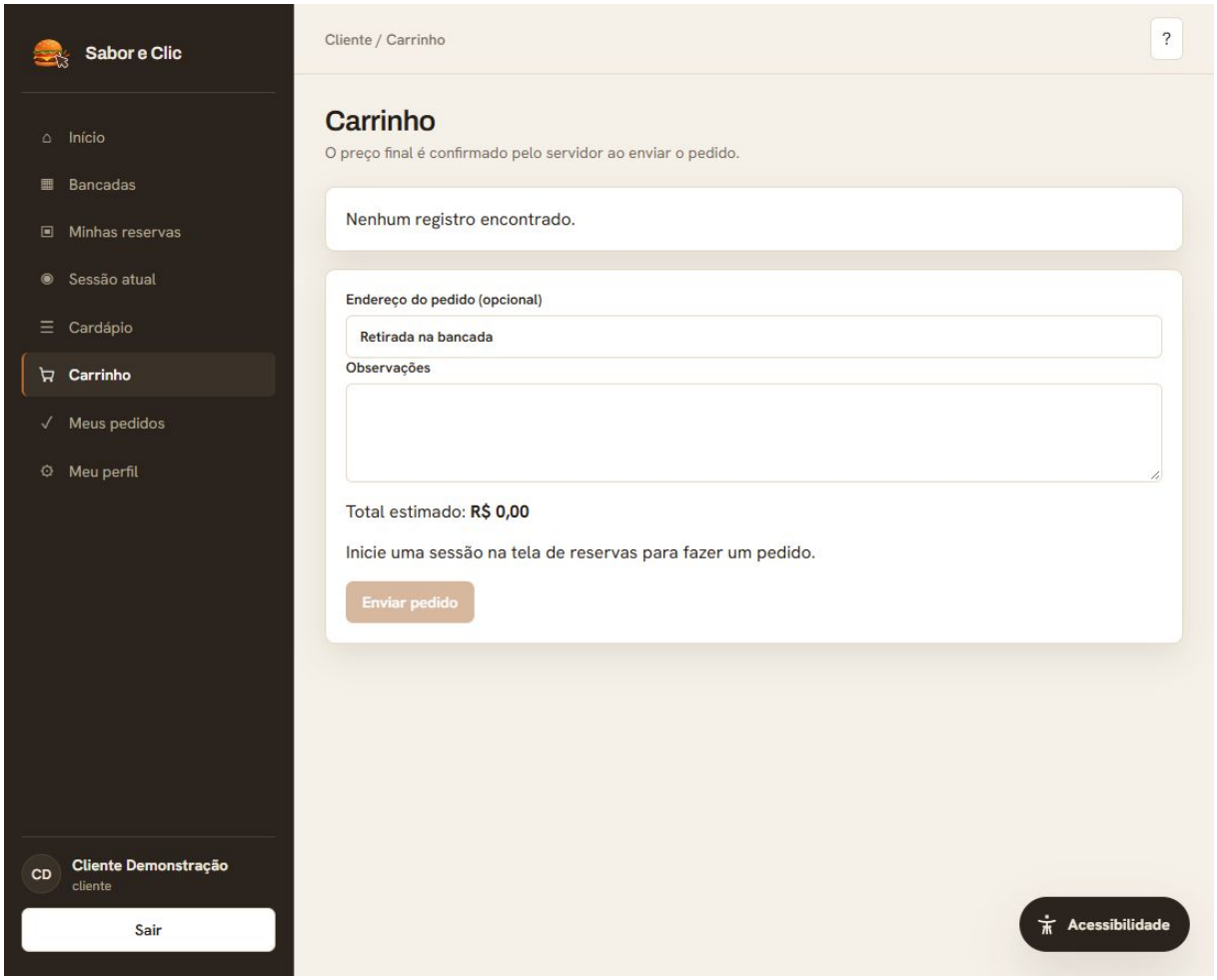


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Carrinho

Permite revisar itens, quantidades, endereço e observações antes do envio.

Figura 17 – Cliente · Carrinho

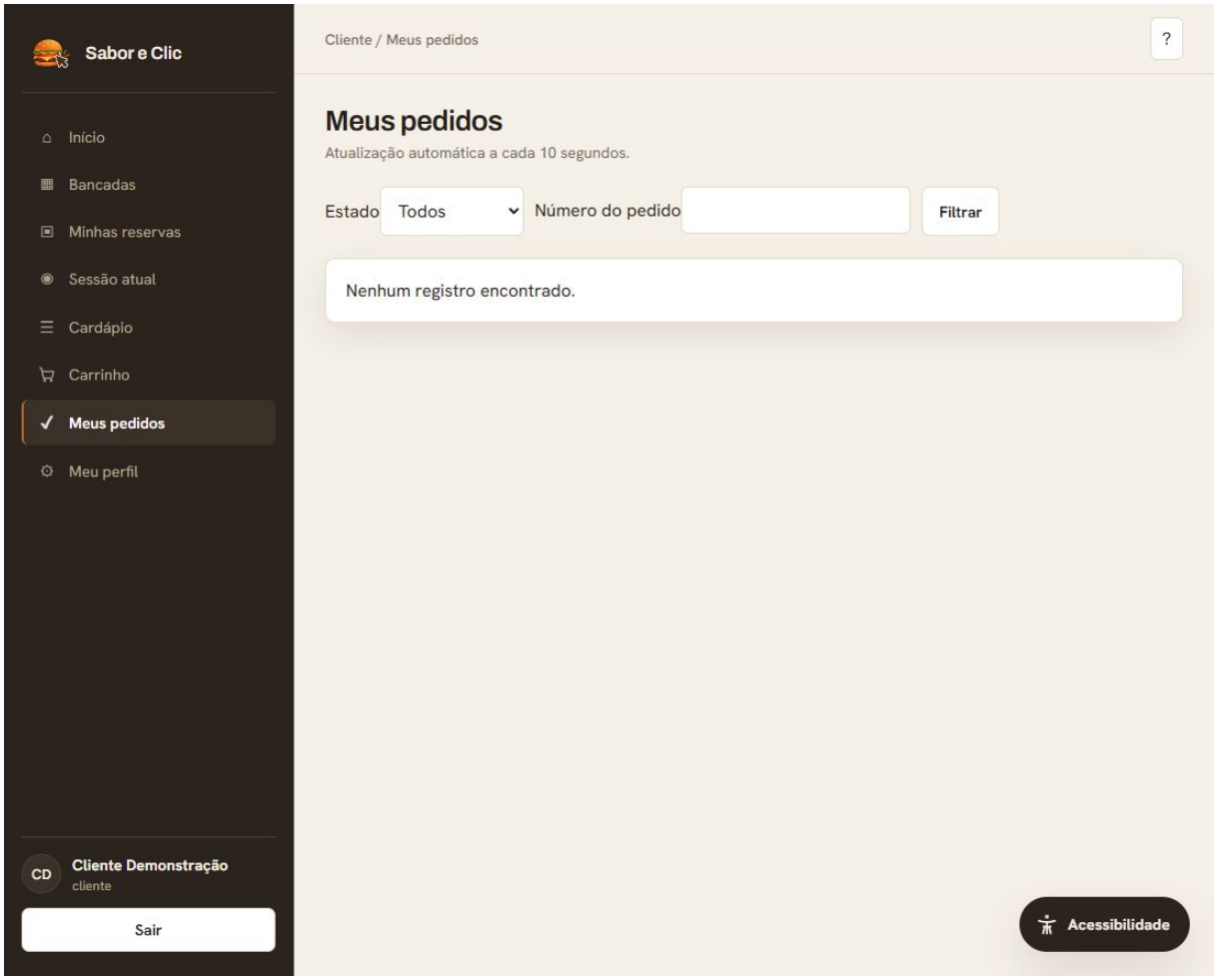


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Meus pedidos

Exibe itens, total e evolução dos pedidos enviados.

Figura 18 – Cliente · Meus pedidos

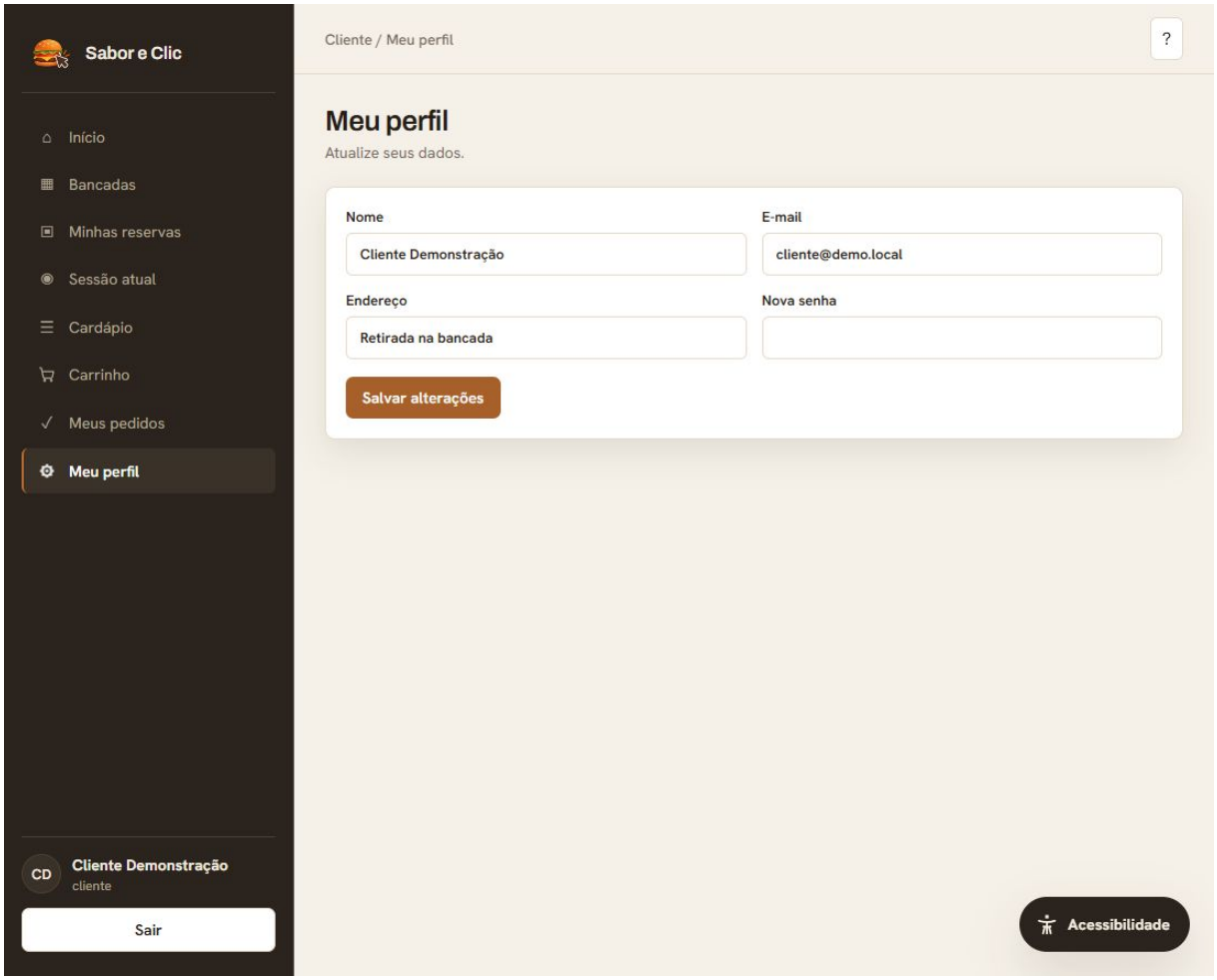


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cliente · Meu perfil

Permite consultar e atualizar os dados da própria conta.

Figura 19 – Cliente · Meu perfil

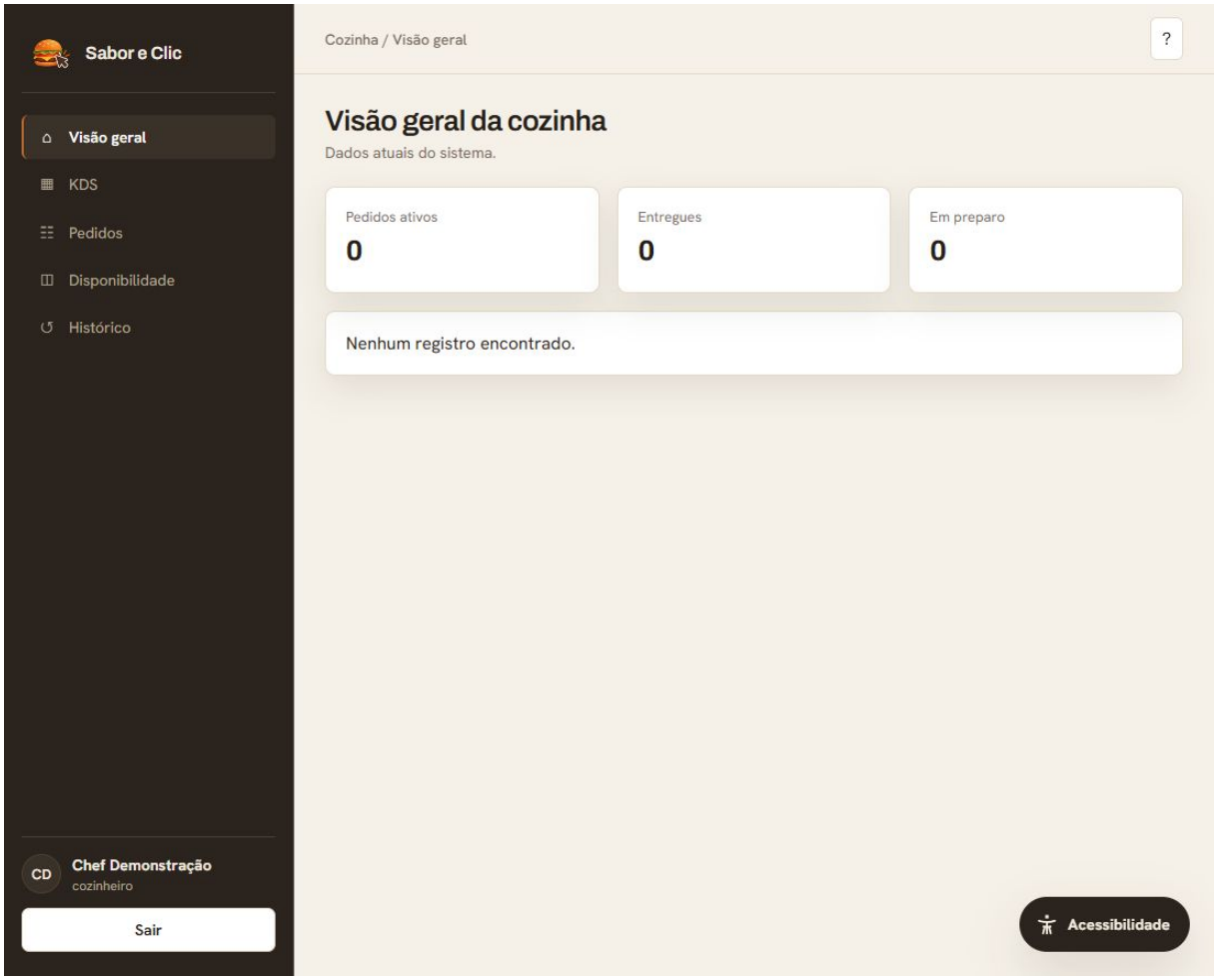


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cozinha · Visão geral

Resume a operação com indicadores dos pedidos.

Figura 20 – Cozinha · Visão geral

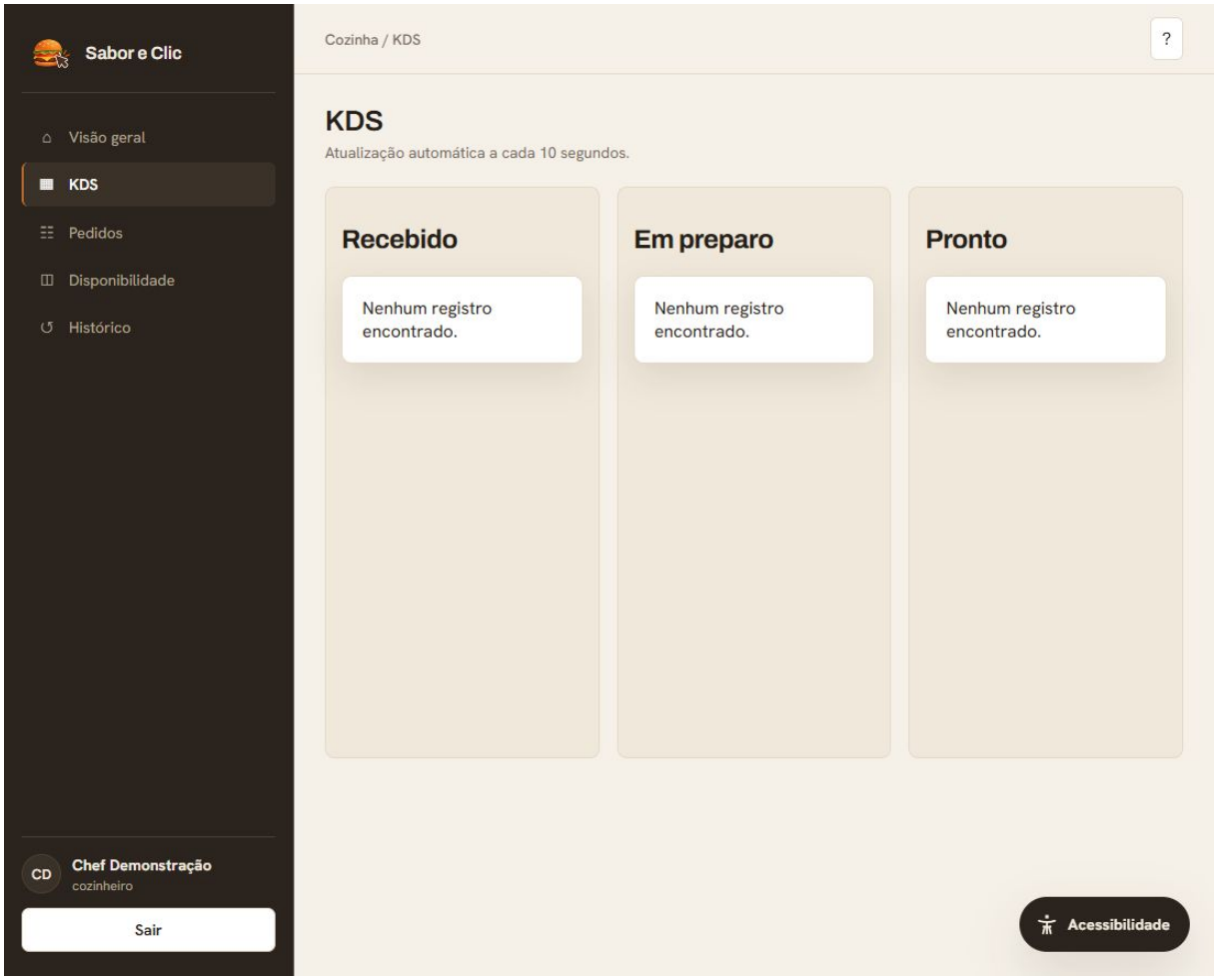


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cozinha · KDS

Organiza os pedidos nas colunas Recebido, Em preparo e Pronto.

Figura 21 – Cozinha · KDS

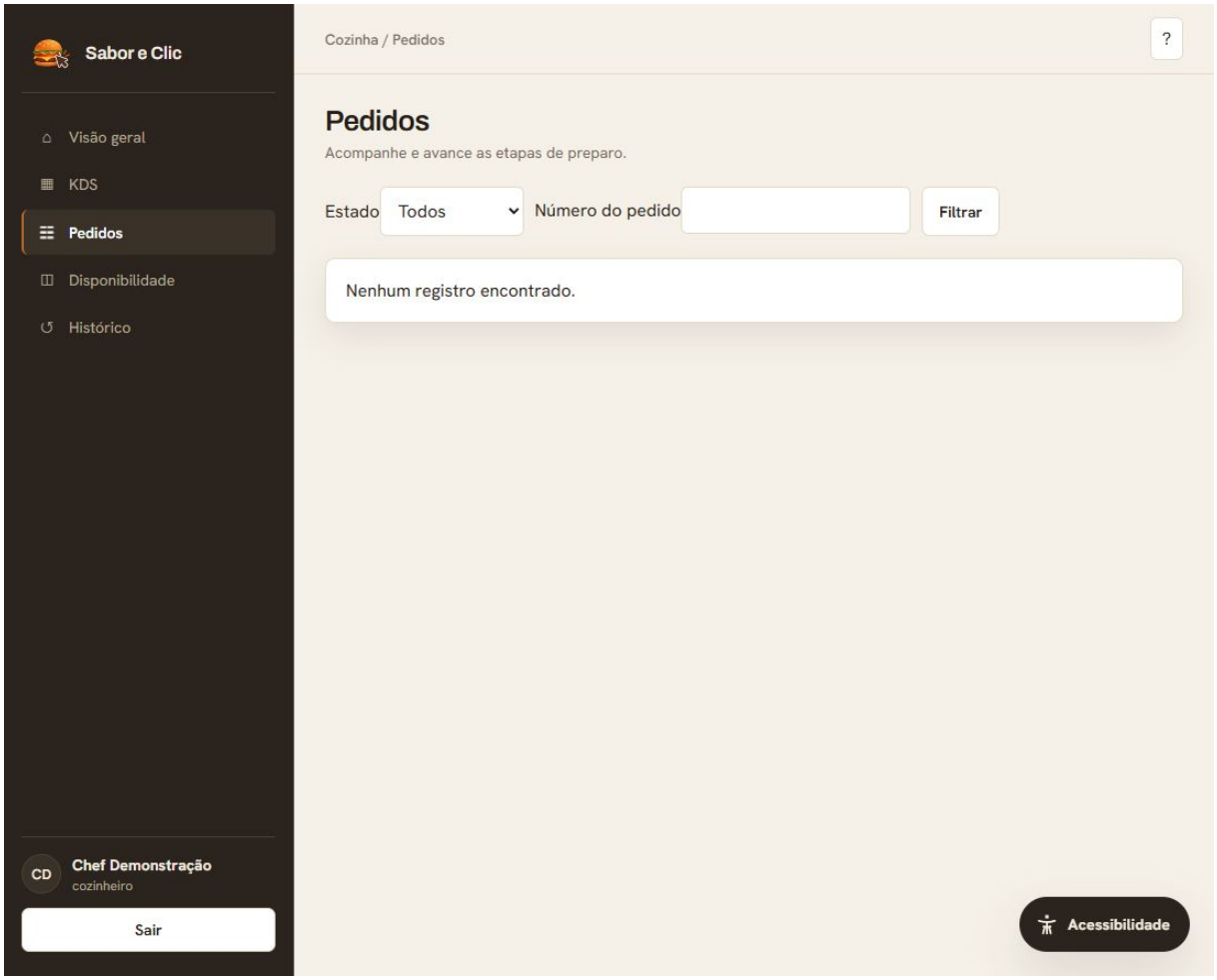


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cozinha · Pedidos

Oferece uma visão em lista com filtros para a operação.

Figura 22 – Cozinha · Pedidos

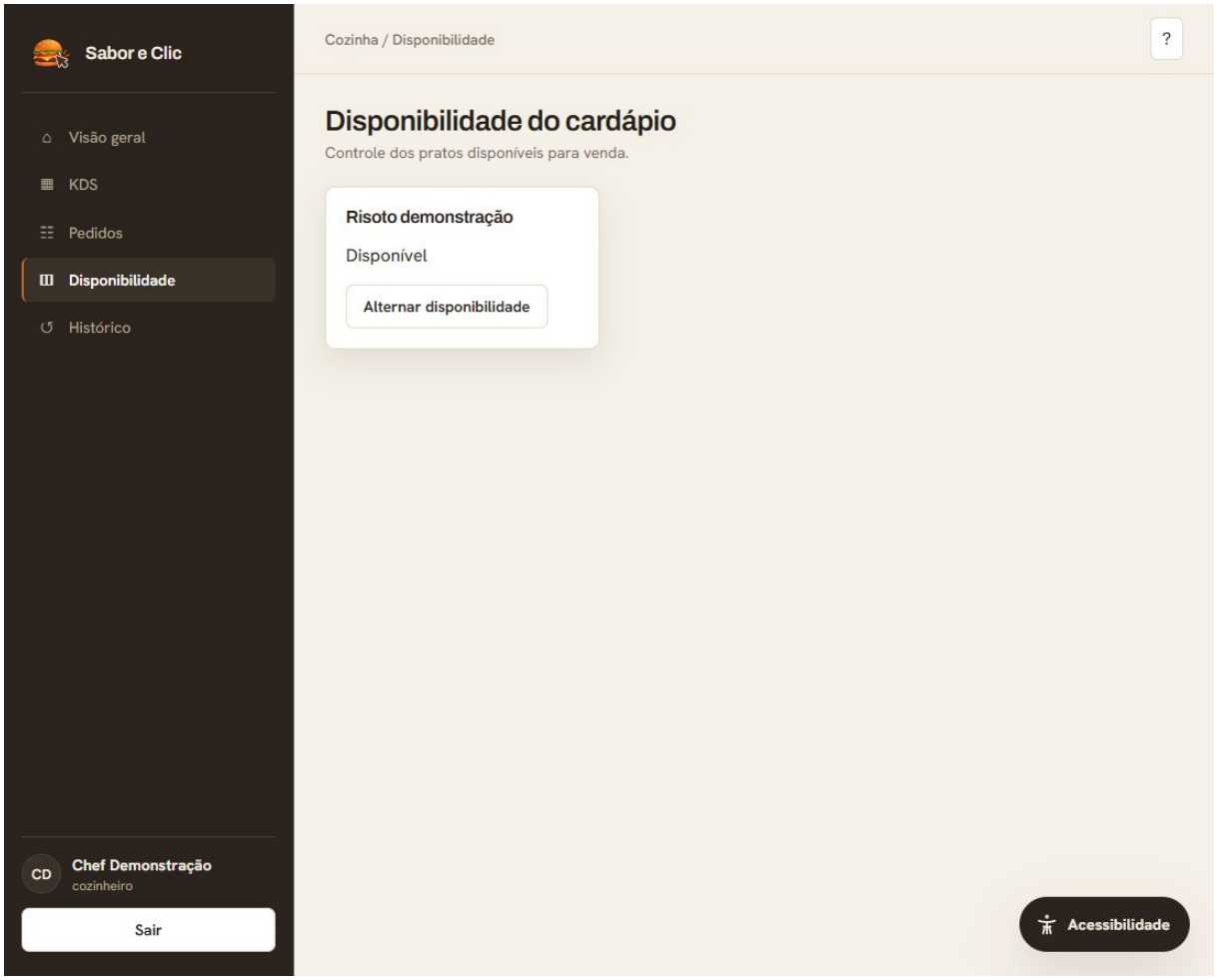


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cozinha · Disponibilidade

Permite consultar pratos e alterar sua disponibilidade no cardápio.

Figura 23 – Cozinha · Disponibilidade

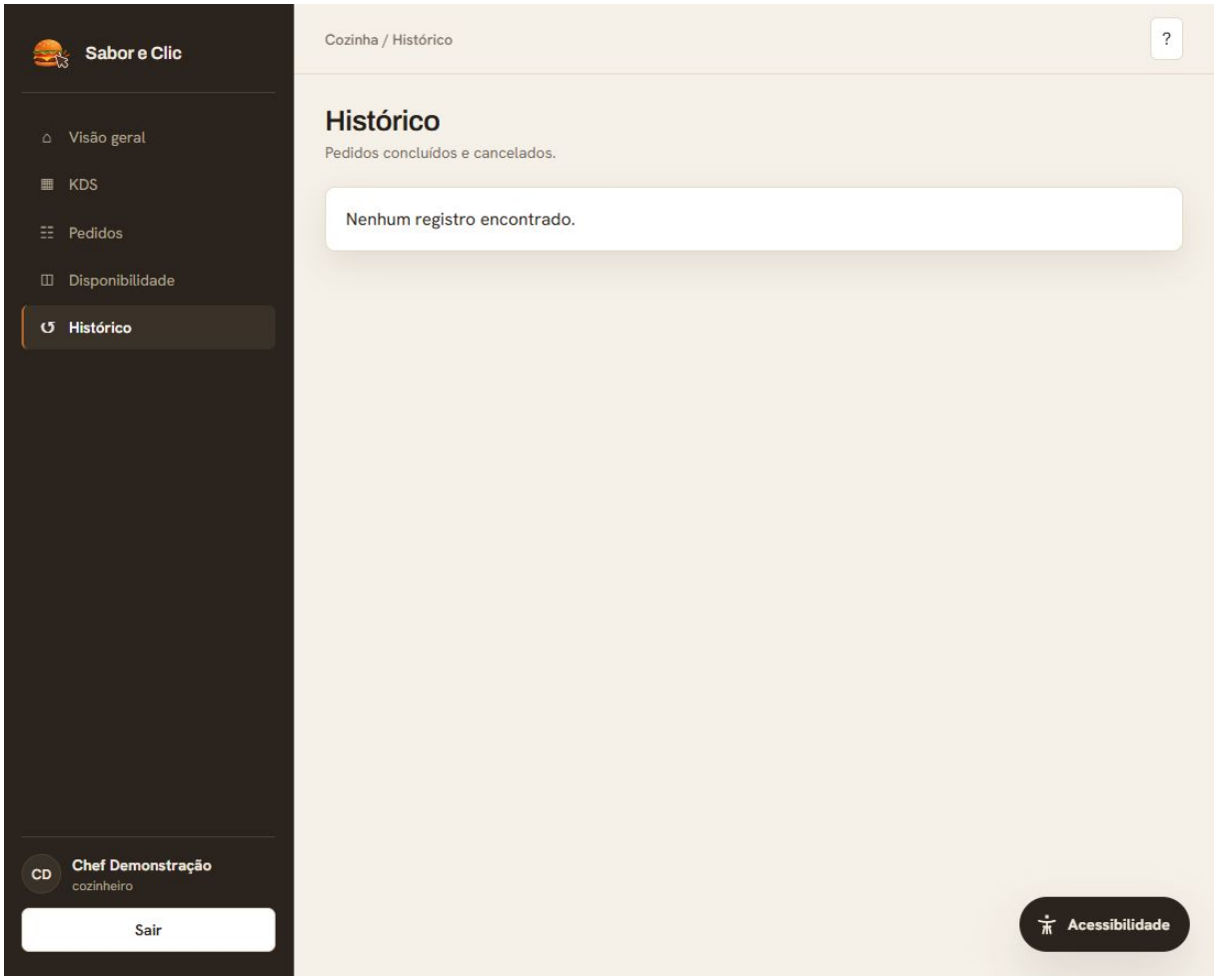


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Cozinha · Histórico

Reúne pedidos finalizados para consulta.

Figura 24 – Cozinha · Histórico

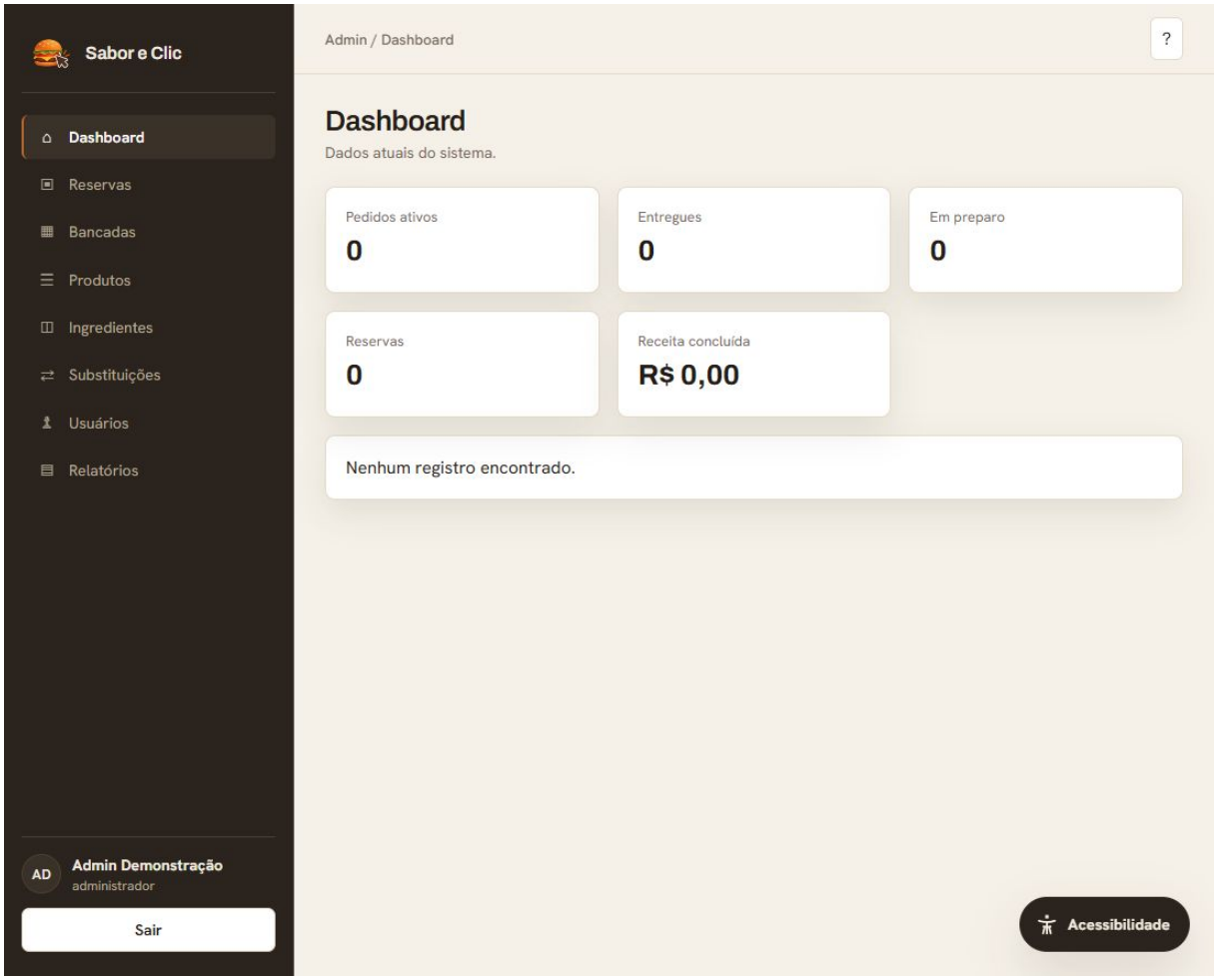


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Painel

Apresenta indicadores gerais da operação.

Figura 25 – Administração · Painel

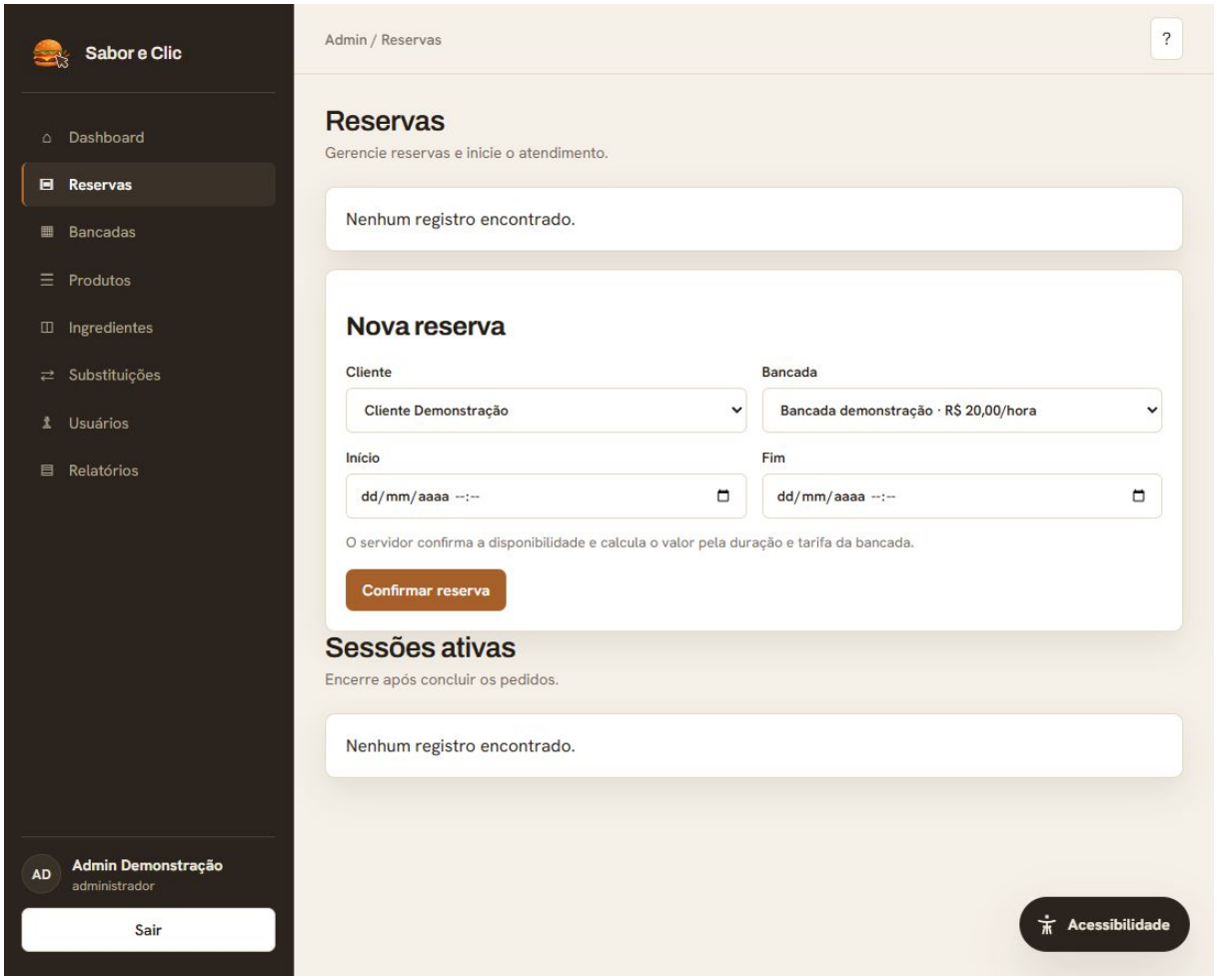


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Reservas

Centraliza a consulta e a gestão das reservas.

Figura 26 – Administração · Reservas

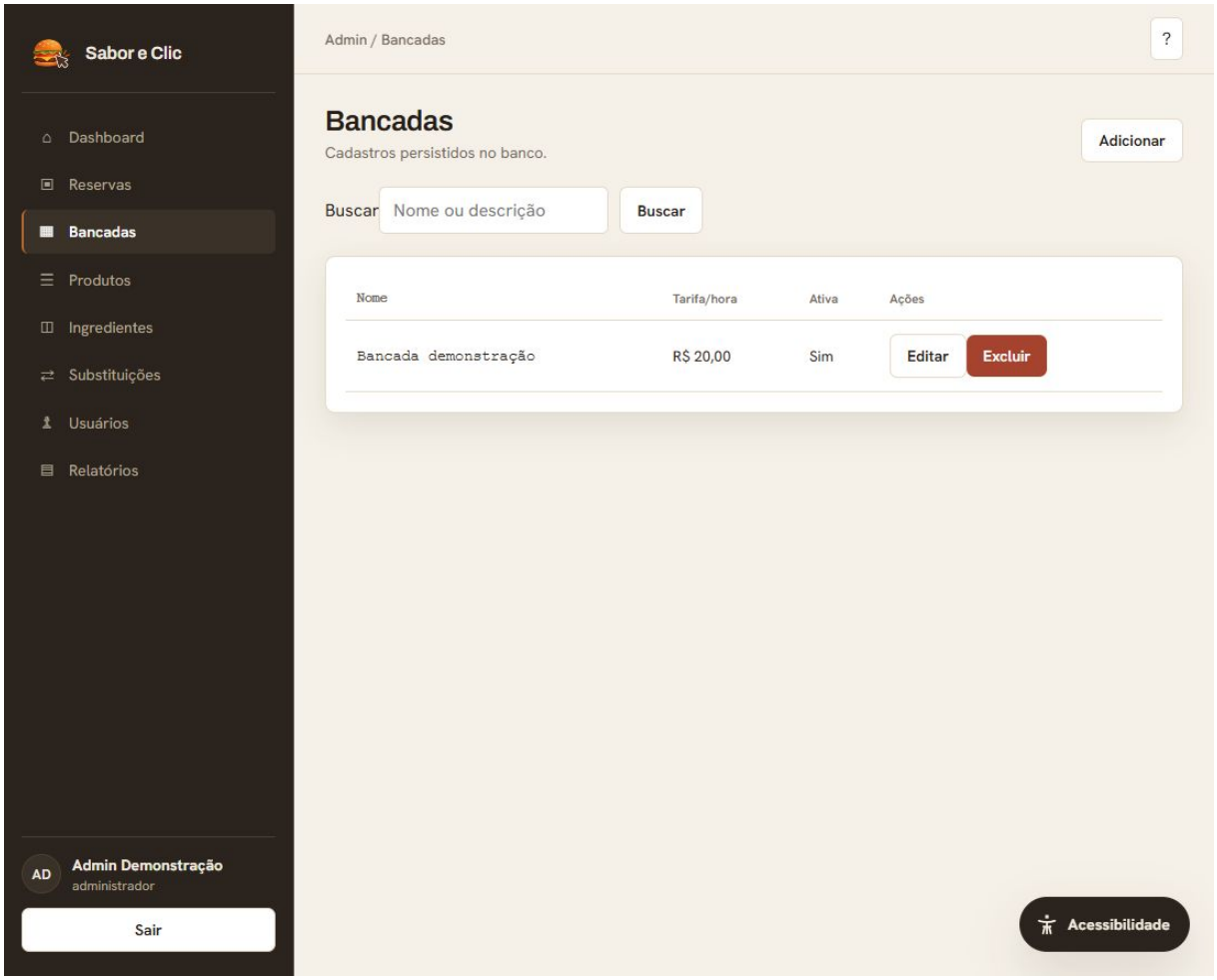


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Bancadas

Gerencia espaços reserváveis, tarifas e situação.

Figura 27 – Administração · Bancadas

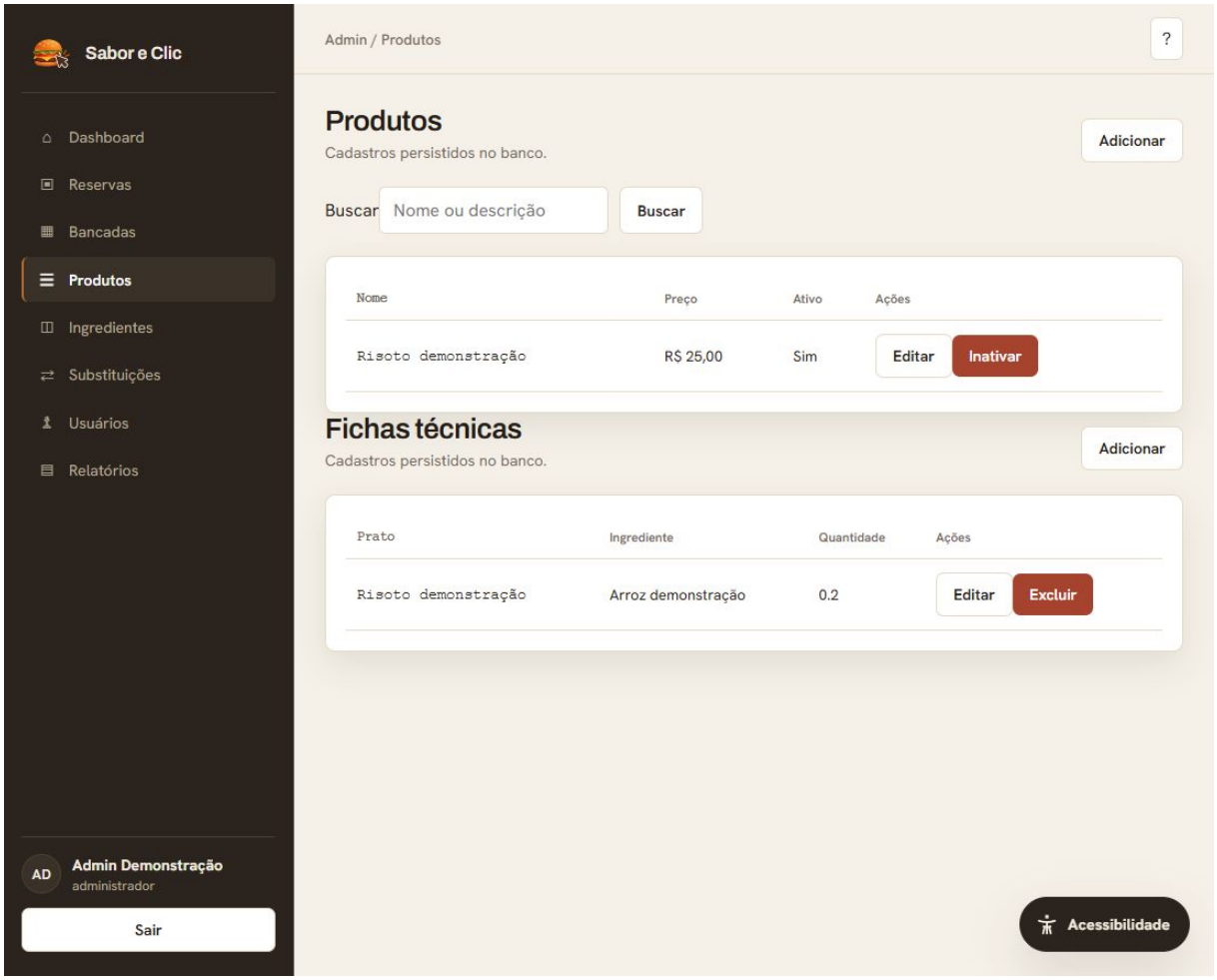


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Produtos e pratos

Gerencia o catálogo e as fichas técnicas usadas no consumo de estoque.

Figura 28 – Administração · Produtos e pratos

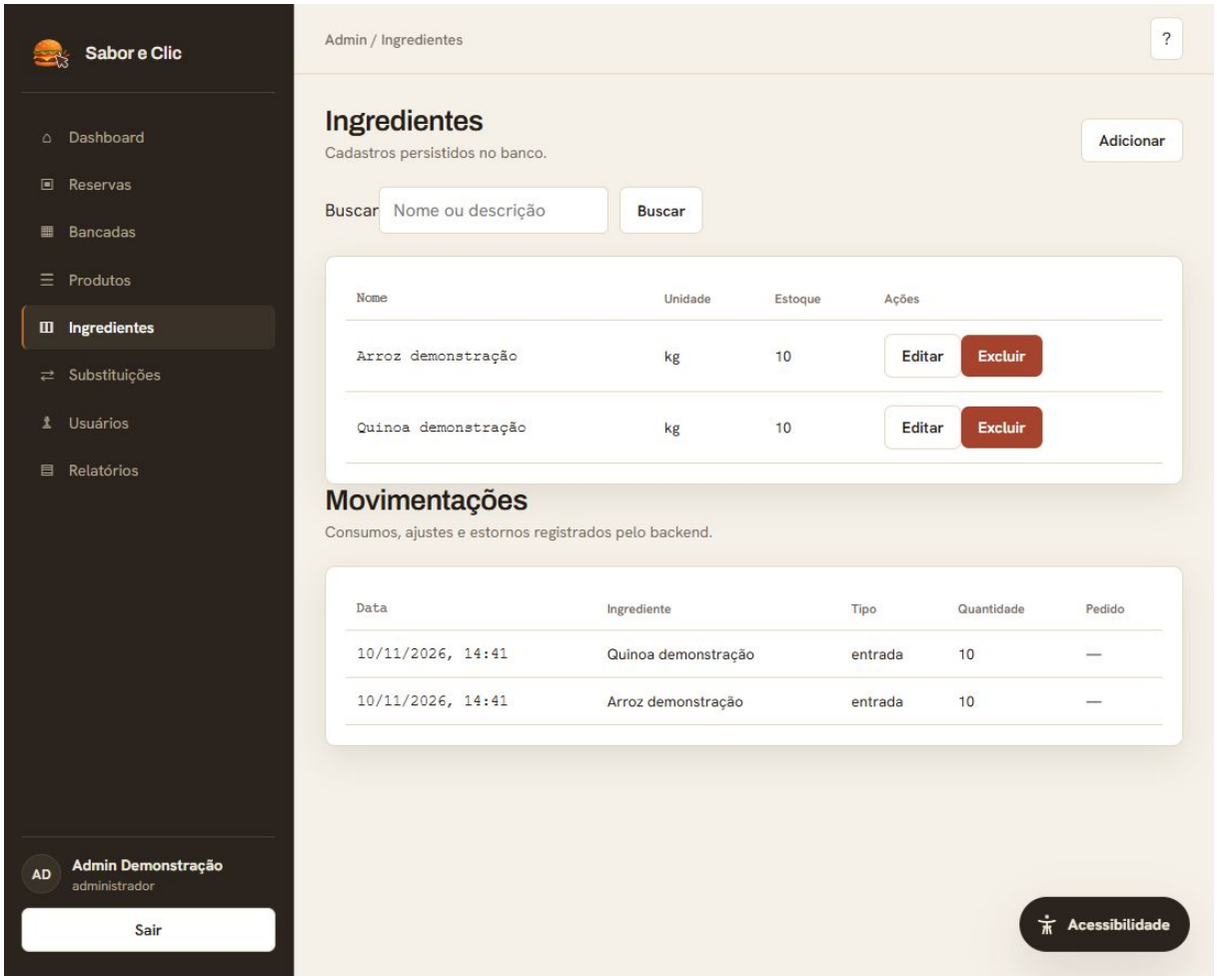


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Ingredientes

Exibe insumos, unidades, saldo e movimentações.

Figura 29 – Administração · Ingredientes

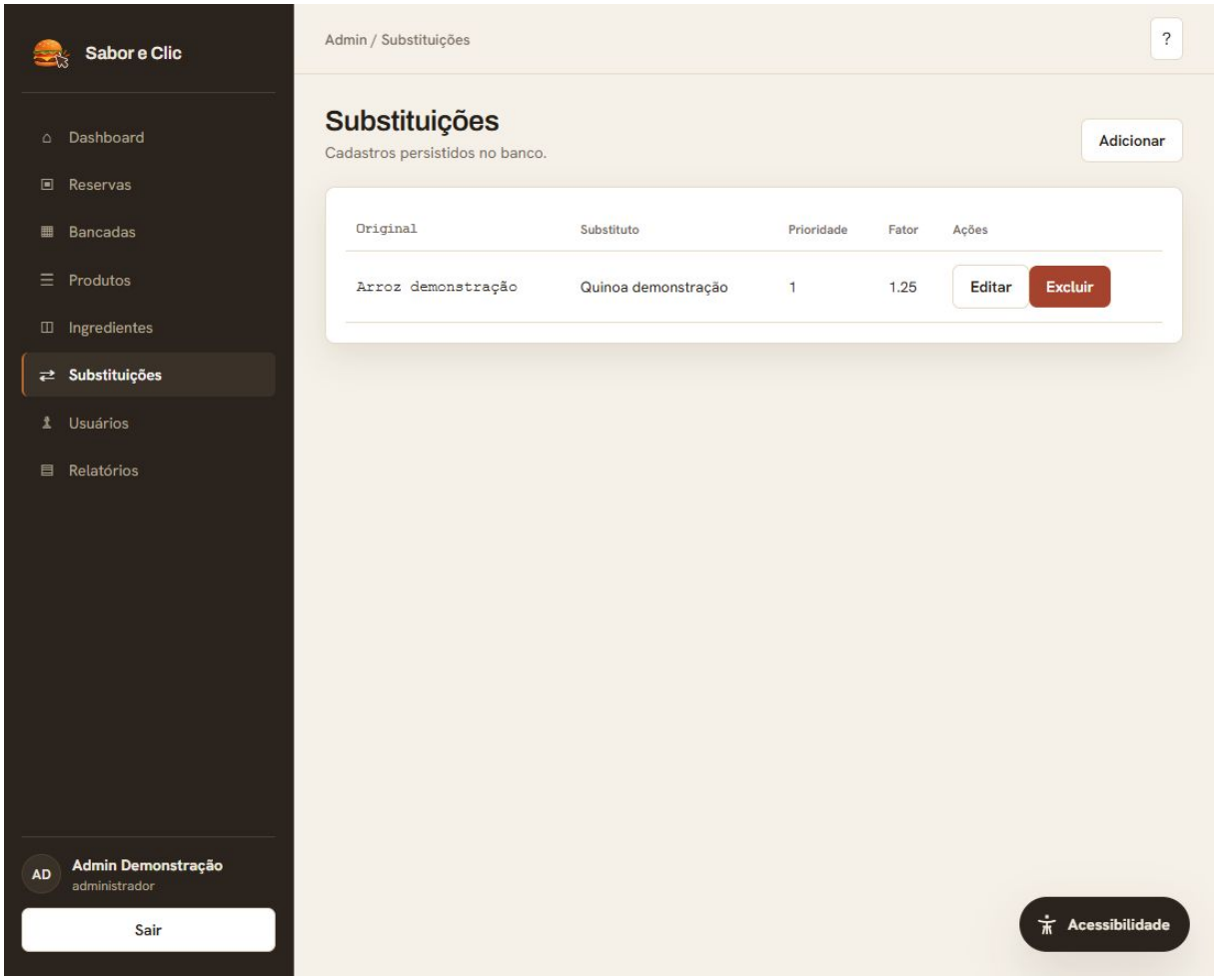


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Substituições

Gerencia alternativas, prioridades e fatores de conversão.

Figura 30 – Administração · Substituições

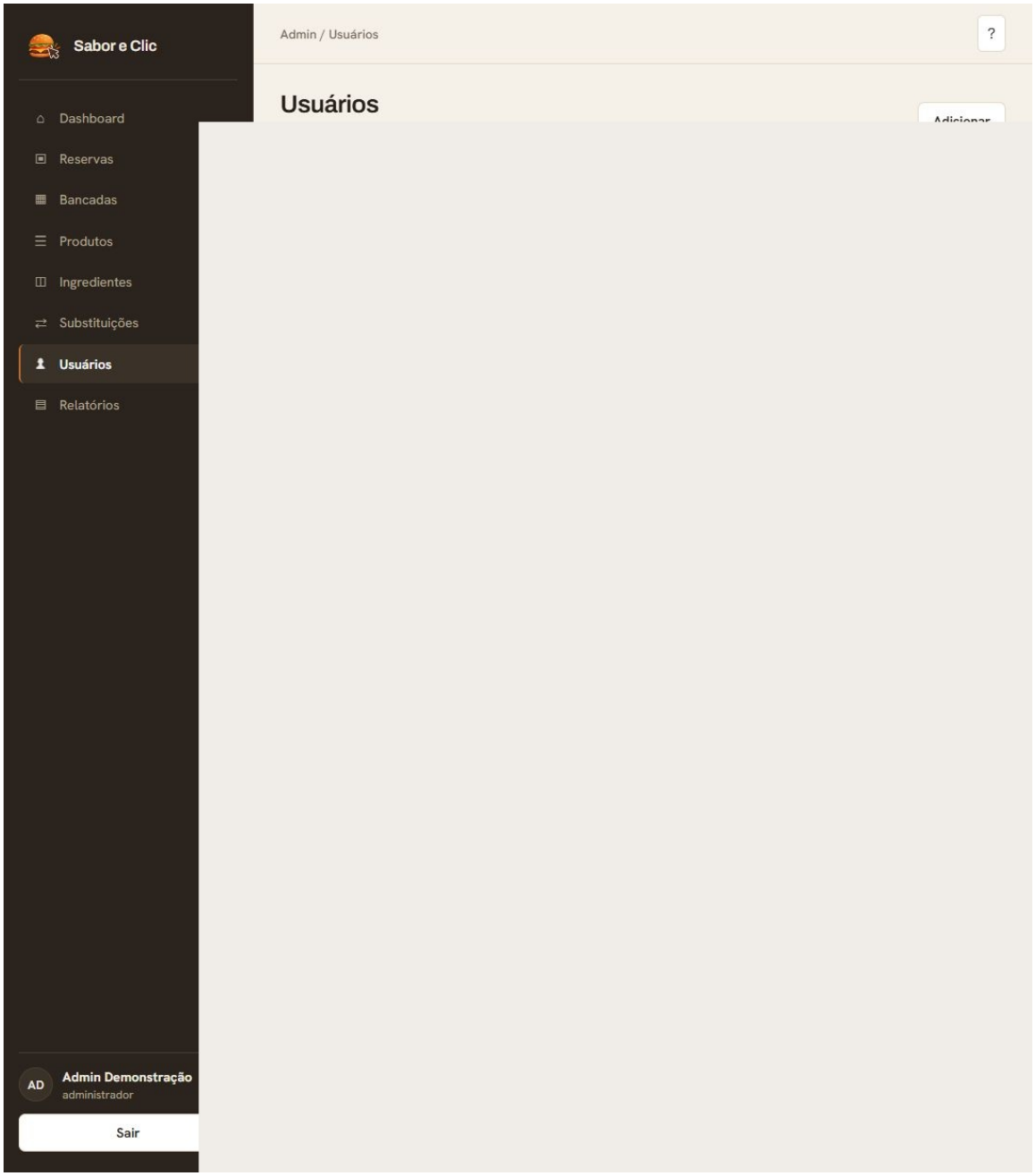


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Usuários

Gerencia contas, perfis e estado de acesso.

Figura 31 – Administração · Usuários

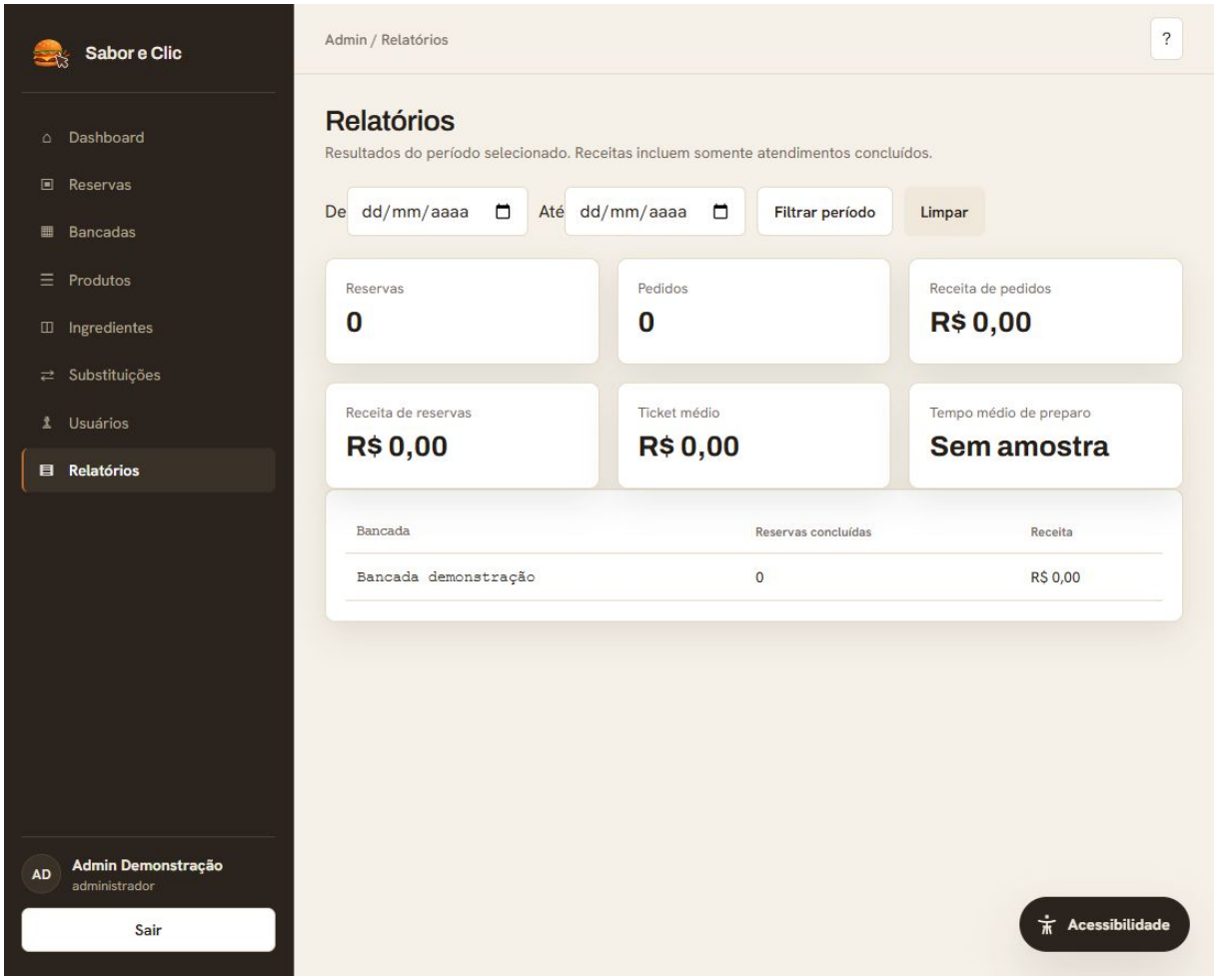


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026). Dados de contas e auditoria ocultados.

Administração · Relatórios

Apresenta indicadores por período e consultas gerenciais.

Figura 32 – Administração · Relatórios

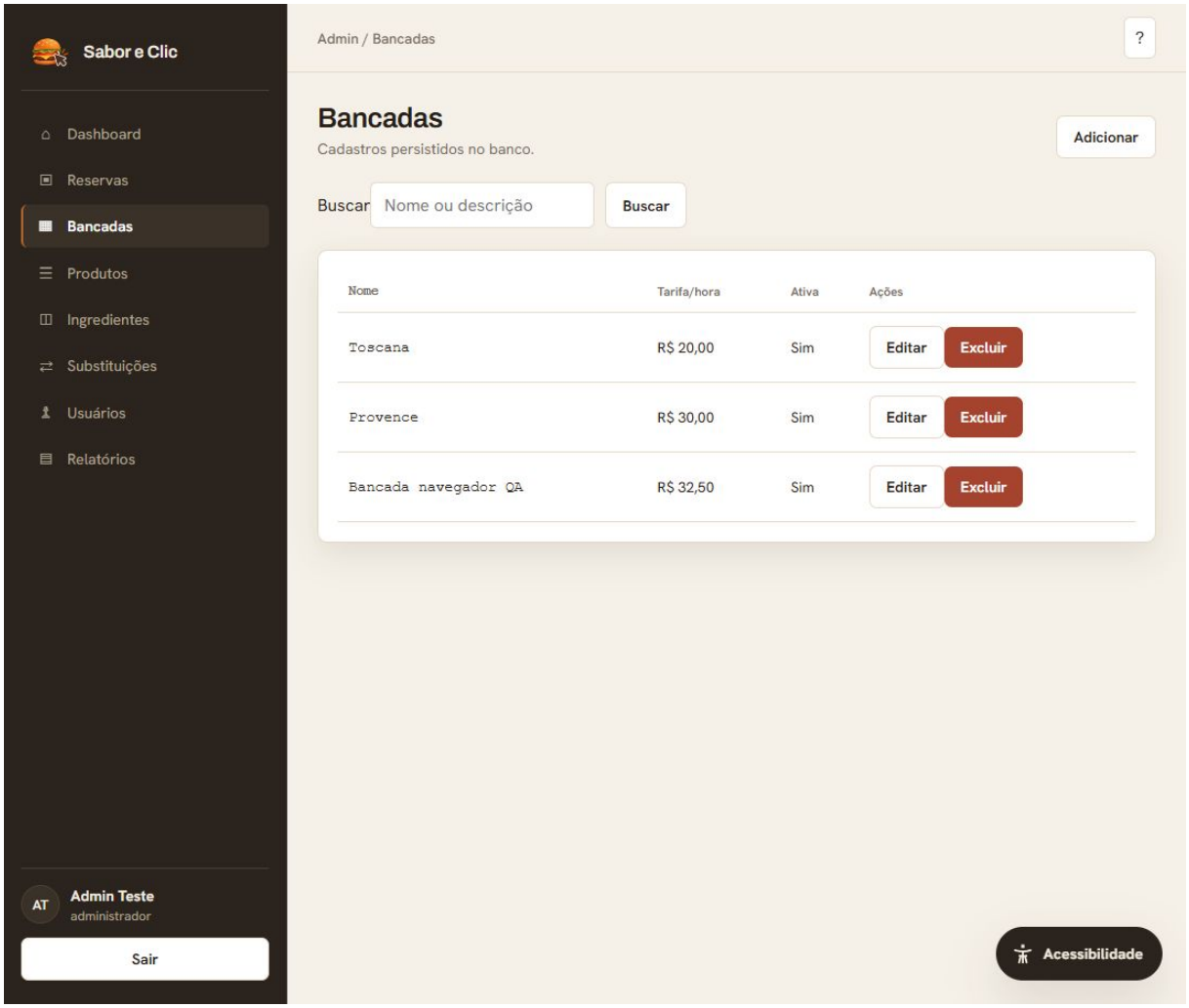


Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).

Administração · Cadastro de bancada

A evidência mostra o cadastro de bancada exercitado no navegador.

Figura 33 – Administração · Cadastro de bancada



Fonte: captura do ambiente de demonstração do projeto Sabor e Clic (2026).